Online Synthesis of MEDA Biochips with Area and Reliability-Aware Module Placement using Chamber-Less Virtual Topology

TAMAL MANDAL, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, India
DEBRAJ KUNDU, Chair of EDA, Technical University of Munich, Munich, Germany
SUDIP ROY, Department of Computer Science and Engineering, IIT Roorkee, Roorkee, India

Real-time execution of bioassays on microelectrode dot array (MEDA) biochips can revolutionize point-of-care diagnostics. Despite numerous design automation efforts for digital microfluidic biochips (DMFBs), there is no online synthesis framework for their advanced counterpart, MEDA biochips, which have various operational advantages. The DMFB-specific virtual topology *[Grissom et al., TCAD, 2014]* is inadequate for MEDA synthesis as it suffers from significant chip-area fragmentation issues. In this article, we propose a virtual topology that provides an abstraction of the physical geometry of MEDA biochips, thereby reducing the algorithmic runtime for MEDA synthesis by reducing the search space for optimization problems. On top of this, we propose an Area-efficient Online Placer (*AOP*) to achieve placement results with minimum chip-area. However, due to the overuse of some microelectrodes, they may be degraded, which may result in unreliable outcomes of the bioassays. Placement Reliability Enhancer (*PRE*) is proposed to improve the reliability of MEDA chips. Finally, a Fast Online Synthesis (*FOS*) is proposed for MEDA based on the virtual topology and these two methods, which always outperform the existing online synthesis in the average assay completion time and the number of routing sub-problems. Simulations confirm that *AOP* significantly reduces both occupied area and algorithmic runtime compared with state-of-the-art *WLSM* and *RLPM*, albeit at the expense of reliability. By integrating *PRE*, *AOP+PRE* achieves reliability similar to *WLSM* and *RLPM*, but requires additional chip area compared with *AOP* while maintaining similar runtime efficiency. Hence, *AOP+PRE* enables efficient and reliable online synthesis for MEDA biochips.

CCS Concepts: • **Hardware → Electronic design automation**; **Emerging technologies;**

Additional Key Words and Phrases: MEDA biochips, digital microfluidics, virtual topology, online synthesis, module placement

ACM Reference Format:
Tamal Mandal, Debraj Kundu, and Sudip Roy. 2025. Online Synthesis of MEDA Biochips with Area and Reliability-Aware Module Placement using Chamber-Less Virtual Topology. *ACM Trans. Des. Autom. Electron. Syst.* 31, 2, Article 24 (December 2025), 29 pages. https://doi.org/10.1145/3774890

Authors' Contact Information: Tamal Mandal, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, India; e-mail: t_mandal@cs.iitr.ac.in; Debraj Kundu, Chair of EDA, Technical University of Munich, Munich, Germany; e-mail: debraj.kundu@tum.de; Sudip Roy (corresponding author), Department of Computer Science and Engineering, IIT Roorkee, Roorkee, UK, India; e-mail: sudip.roy@cs.iitr.ac.in.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 1084-4309/2025/12-ART24
https://doi.org/10.1145/3774890

1 Introduction

Over the past two decades, microfluidic biochips have undergone a remarkable evolution, marked by significant advancements [1] in accuracy, efficiency, feasibility, scalability, and sensor integration capabilities, enabling the execution of complex bioassays on-chip. This rapid progression has transformed biochip technology, elevating its potential and applications, such as RT-PCR and ChIP bioassay [2]. A sequence of various fluidic operations is called a bioassay, generally represented as a **directed acyclic graph** (**DAG**). A synthesis methodology is required to execute a bioassay on a physical biochip. Numerous offline synthesis methodologies have been proposed, including scheduling [3, 4], placement [2, 5, 6], and routing [7], to perform static compilation [8] of bioassay. Offline synthesis methodologies focus on generating optimized designs (scheduling, placement, and routing results) with less emphasis on algorithmic runtime. In this case, synthesis is performed only once during the entire lifespan of the chip. The compiled executable binary is packaged into an application-specific device, eliminating the need for repetitive synthesis to execute the same bioassay repeatedly. However, online synthesis [9–11] dynamically manages on-chip resources, creating a flow akin to a nonlinear state machine. Online synthesis emphasizes algorithmic runtime and enables real-time compilation, as the end-user must wait for synthesis completion before executing each bioassay. A **Just-In-Time** (**JIT**) compilation framework [12] provides flexible and real-time bioassay execution, introducing end-user programmability and leveraging the limited cyber-physical capabilities of conventional DMFBs. Deterministic execution is unnecessary for it; instead, execution can be dynamically driven by live sensory feedback. Online synthesis methodologies are crucial for incorporating control flow information into the synthesis process. Figure 1 highlights the tradeoff between offline and online synthesis for biochips.

Several sample preparation methodologies [13, 14] have been proposed in the literature to efficiently generate a mixing graph, which is a sequence of fluid mixing operations that produces the desired volumetric ratio of a given set of reagents. **Digital microfluidic biochip** (**DMFB**) architectures are evolving rapidly, and sample preparation algorithms [15, 16] are also advancing over time to exploit the advanced features of newer biochips. Newer architectures, such as MEDA, enable the synthesis of complex mixing graphs with variable-size mixers, which conventional DMFB architectures cannot achieve. The mixing graph with mixers of variable size produced by [16] is shown in Figure 2(a). In contrast, the corresponding mixing graph that only has two mixers of size, compatible with DMFBs, is shown in Figure 2(b). The enhanced cyber-physical capabilities of MEDA biochips enable integration with other microfluidic devices and sensors, allowing for the construction of a more complex and versatile analytical system. Existing virtual topology and online synthesis methodologies [9–11, 17] are designed for conventional DMFBs. If the same is applied to MEDA biochips, it results in under-utilization of their advanced features, increased occupied area (α), and unnecessarily extended assay completion time (T_c) due to fragmented space. Various non-iterative scheduling [3, 18] and routing [19] methodologies exist for MEDA to develop an online synthesis framework. However, existing placement algorithms for MEDA biochips are either simulated annealing-based or learning-based, both of which are inadequate for online synthesis. The proposed virtual topology-based **area-efficient online placer** (***AOP***) and **placement reliability enhancer** (***PRE***) integrate seamlessly with the existing non-iterative scheduler and router, providing a real-time online synthesis framework that offers better end-user programmability.

The key contributions of this article are as follows:

— A chamber-less, row-based virtual topology is proposed for MEDA biochips that provides a real-time assay execution environment without any fragmented area.

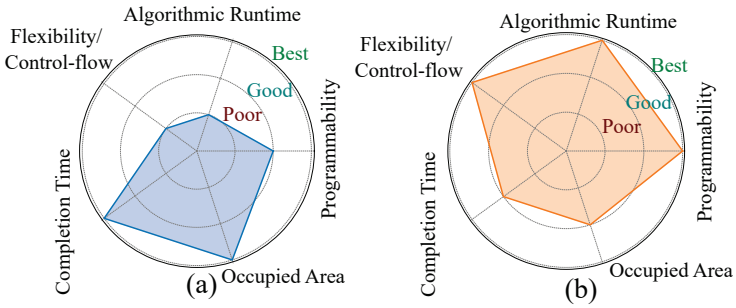


Fig. 1. Tradeoff between (a) offline and (b) online synthesis [9].

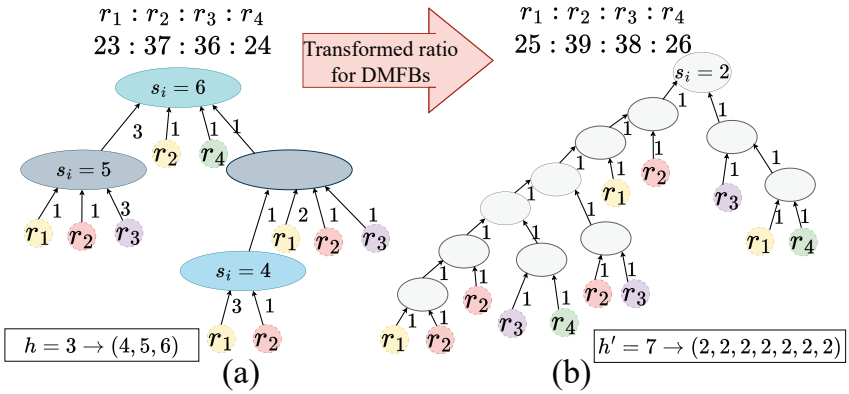


Fig. 2. For ratio 23 : 37 : 36 : 24, (a) mixing graph with variable size mixers (to be implemented on MEDA) and (b) the transformed mixing graph with mixer size 2 (to be implemented on DMFB).

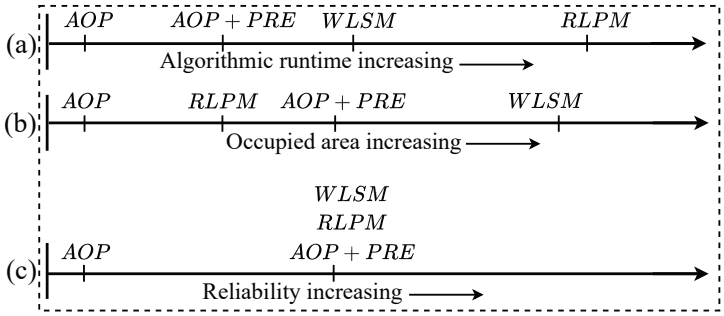
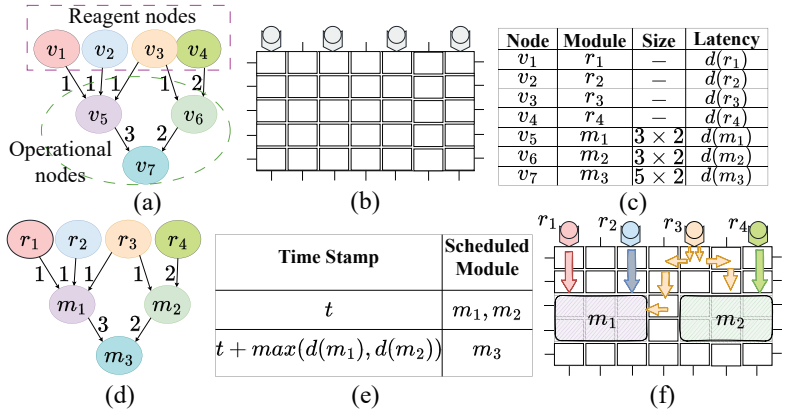


Fig. 3. Relative positions of two existing (*WLSM* [5], *RLPM* [6]) and two proposed (*AOP* and *AOP+PRE*) module placement methods for MEDA synthesis in optimizing performance parameters: (a) algorithmic runtime, (b) occupied area and (c) reliability.

— A virtual topology-based Area-efficient Online Placer (*AOP*) is proposed to get area-efficient placement results.
— A Placement Reliability Enhancer (*PRE*) is proposed to provide reliable placement solutions if reliability becomes more crucial than the occupied chip-area.



Node	Module	Size	Latency
v_1	r_1	–	$d(r_1)$
v_2	r_2	–	$d(r_2)$
v_3	r_3	–	$d(r_3)$
v_4	r_4	–	$d(r_4)$
v_5	m_1	3×2	$d(m_1)$
v_6	m_2	3×2	$d(m_2)$
v_7	m_3	5×2	$d(m_3)$

Time Stamp	Scheduled Module
t	m_1, m_2
$t + max(d(m_1), d(m_2))$	m_3

Fig. 4. For (a) a mixing graph implemented on (b) a 5×7 MEDA chip, (c) node-module binding, (d) mixing graph after module binding, (e) scheduling result, and (f) placement and routing instances.

— Finally, we present a Fast Online Synthesis (*FOS*) framework to enable real-time compilation on MEDA biochips.

Overall, Figure 3 clearly explains how the proposed methods *AOP* and *AOP+PRE* stand relatively with two existing module placement methods [5, 6] for MEDA synthesis to optimize three performance parameters (algorithmic runtime, chip-area, and reliability).

The remainder of the article is organized as follows. Section 2 describes the background and related prior work, whereas Section 3 describes the modeling of virtual topology, and Section 4 illustrates the motivation of the proposed work. Problem formulation is described in Section 5, and the proposed methodology is discussed in Section 6. Simulation results are provided in Section 7, and finally, Section 8 concludes the work.

2 Backgrounds and Prior Work

2.1 Backgrounds

Micro-Electrode-Dot-Array (MEDA) biochips are advanced devices for microscale fluid manipulation used in medical diagnostics, drug development, and biochemical research. A 2D grid of electrodes that controls discrete droplet movement via electrode actuation sequences enables various fluidic operations such as droplet transportation, merging, splitting, mixing, dispensing, and sensing. A MEDA architecture is shown in Figure 4(b), a collection of 5×7 microelectrodes and 4 reservoirs. MEDA biochips [20] offer finer granularity, variable mixer sizes, and precise volume control, requiring lower voltages for actuation [21]. Unlike simple DMFBs, MEDA biochips are more reliable due to their selective sensing capabilities [22].

A bioassay is represented as DAG, where nodes represent fluidic operations and edges signify the dependencies among those operations. Figure 4(a) shows a mixing graph with 4 dispensers and 3 mixing nodes. In module binding, each fluidic operation must be mapped to an on-chip module in the module library (M_{lib}). Figure 4(c) shows the module binding, where 4 dispense nodes are mapped to 4 reservoirs r_1, r_2, r_3, r_4 and 3 mixing nodes are mapped to 3 mixing modules m_1, m_2, and m_3. After binding the fluidic operations to the on-chip modules, the mixing graph is shown in Figure 4(d). Scheduling determines the start and end times of each operation, aiming to minimize either scheduling length (T_l), which affects the completion time (T_c), or the resources (number of mixers) while maintaining dependencies.

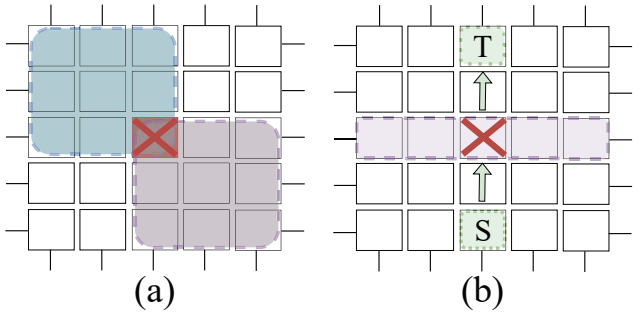


Fig. 5. (a) Placement failure [23], and (b) routing failure [23].

Placement follows scheduling and determines the physical location of each module on the reconfigurable biochip. Such instances of placement for modules m_1 and m_2 are shown in Figure 4(f). Placement [2] aims to provide a routable placement solution with minimum area. Microelectrodes can be damaged from repeated use, which can impact the reliability of the biochip. In recent literature, a class of reliability-aware placement solutions [5, 6] has been proposed to maintain the reliability of MEDA biochips. Reliability can be enhanced at the cost of the occupied chip-area. The placement solution [6] aims to optimize both the chip-area and the overlapping value (Φ, representing the maximum number of modules that use a specific microelectrode for placement). Higher reliability comes with a lower Φ value. This learning-based placer [6] provides good quality solutions; however, it suffers from a high algorithmic runtime, which is a potential drawback for developing an online synthesis framework.

Routing is required when the output of one module is used as input in another module, while they are placed in different locations. Routing is also needed to transport fluid from the reservoir to the modules. An instance of routing is illustrated in Figure 4(f), where the fluid is routed from the reservoir to the mixing modules m_1 and m_2. Each droplet that requires routing is referred to as a routing sub-task. The job is to find the shortest path between the source and destination of each routing sub-task and then perform parallel routing among sub-tasks without collision. A fast Router [19] finds the shortest Manhattan path using the maze routing principle and then performs collision-free parallel droplet routing among sub-tasks. In addition to Manhattan distance movement, diagonal droplet movement, and droplet morphing are supported [7] on MEDA biochips to perform more efficient on-chip droplet routing.

2.2 Related Prior Work

In the literature of microfluidic biochips, two types of offline synthesis are predominantly discussed: (i) sequential synthesis, and (ii) one-pass synthesis, which was proposed with the argument that sequential synthesis may lead to design gaps, as illustrated in Figure 5 [23]. Figure 5(a) shows the placement failure where on a $5 \times 5 = 25$ size grid two $(3 \times 3) = 9$ cells requirement cannot be made. However, at the time of scheduling, it is not possible to detect the placement failure. Once it fails, we need to start again from the beginning. Another issue is routing failure; if the same microelectrodes are used for both module placement and routing, then there is a possibility of routing failure due to the dynamic placement of modules. Figure 5(b) shows a valid module placement; however, routing cannot be done until the placed module finishes its operation, which leads to routing failure or starvation. To address these problems, the *EOPS* framework [23–25] treats scheduling, placement, and routing as a single task. One-pass synthesis prevents design gaps

but increases synthesis complexity, reduces flexibility, and compromises robustness and real-time control required for cyber-physical applications.

In static compilation, the basic blocks are compiled in isolation using existing offline scheduling [3, 4], placement [2, 6], and routing [7]. While this approach allows for optimization, it limits cyber-physical adaptability. To handle control flow within the static compilation framework, all possible basic blocks must be compiled before execution [8]. However, since control flow often depends on runtime sensor feedback, many pre-compiled blocks may never be utilized, resulting in wasted computation and inefficiency. In contrast, JIT or dynamic compilation frameworks [12] for DMFBs compile each basic block on the fly at runtime using online synthesis. The basic block to be compiled next depends on the output of the current basic block. This allows real-time sensing to guide the synthesis process, ensuring that only the required basic blocks are compiled and executed. Figure 6(a) shows a BioScript code that represents a bioassay with four sequential (action) basic blocks, B1 to B4, excluding conditional blocks. Figure 6(b) illustrates the corresponding control-flow graph with conditional branching. After executing B1, the next block to be executed depends on the value of *volumeWeight*. In static compilation, all basic blocks (B1–B4) are compiled in advance using an offline synthesis approach before the execution begins. However, in the case of JIT compilation, once the value of *volumeWeight* is obtained at runtime, it determines which basic block among B2, B3, and B4 needs to be compiled next. Then synthesizes only that specific basic block using an online synthesis approach at the backend.

3 Limitations of DMFB-Based Virtual Topology for MEDA Biochips

A MEDA biochip, characterized by its 2D grid of microelectrodes, enables each cell to participate in various fluidic operations, such as mixing, splitting, detection, storage, and transportation [20, 21]. Offline synthesis for predefined bioassays with known conditions on a fixed physical layout allows extensive optimization without real-time adjustments. However, online synthesis requires a different approach due to the unpredictability of the execution environment. Hence, virtual topology is essential for online synthesis, allowing dynamic reconfiguration and real-time adaptability. Virtual topology generally divides the whole chip space into multiple work chambers where all the fluidic operations (including mixing) can be performed. It reduces the placement problem to a work chamber allocation problem. The existing virtual topology design may consist of either homogeneous chambers or heterogeneous chambers. The existing virtual topology [9] designed for DMFB only supports fixed-size mixers, but not for MEDA, which requires variable-sized mixers. LEMMA 1 and LEMMA 2 illustrate the drawbacks of using the existing DMFB-specific virtual topology [9] for MEDA biochips.

Lemma 1. *Internal fragmentation can occur when variable-sized mixers are required to be placed in homogeneous chambers.*

If a mixer module of size $p \times q$ is required to be placed on $\mathcal{K}$ homogeneous chambers of size $a \times b$. Then the internally fragmented (unused) area in the chamber can be calculated as follows:
$(b-q)\cdot p + (a-p)\cdot b = a\cdot b - p\cdot q$, where $(a \geq p) \wedge (b \geq q)$. To ensure valid placements within the existing virtual topology for the largest mixer, the chamber size must be equal to or larger than the size of the largest mixer used. Consequently, when a smaller mixer is placed, the condition $a\cdot b - p\cdot q > 0$ leads to internal fragmentation, as illustrated in Figure 7(a).

A mixing graph with variable-size mixers shown in Figure 8(a), if required to be placed on the existing virtual topology [9] with **homogeneous chambers**, then they suffer from internal fragmentation, as shown in Figure 8(b) at time $t = 1$.

Lemma 2. *External fragmentation can occur when variable-sized mixers are required to be placed in heterogeneous chambers.*

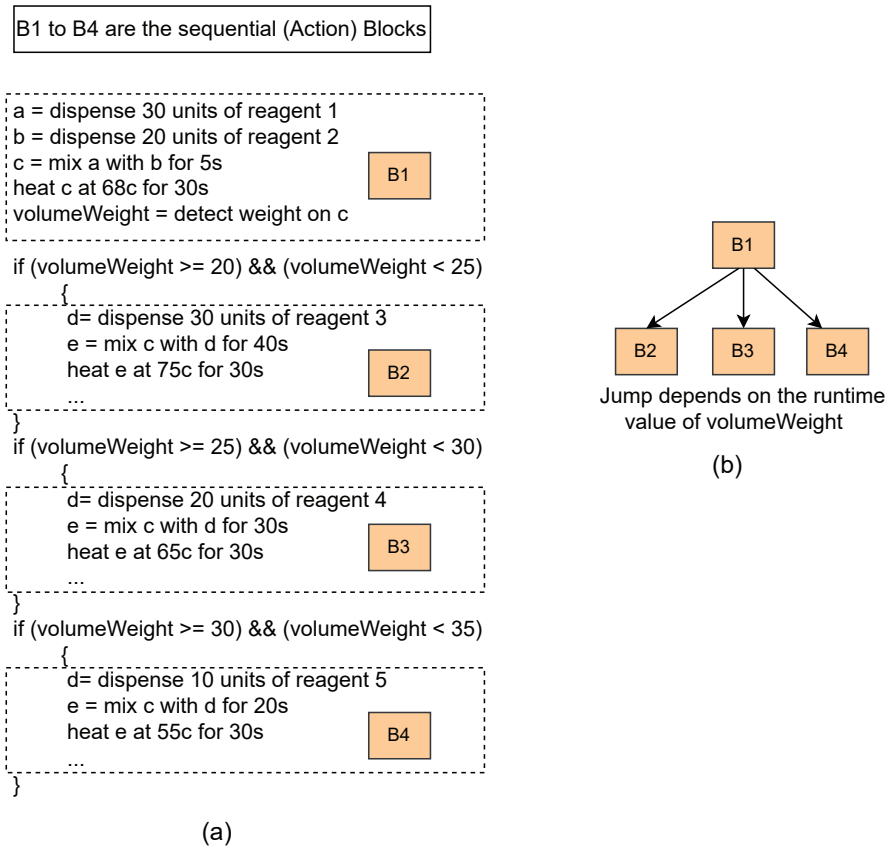


Fig. 6. (a) A bioassay written in bioScript language with basic blocks representation. (b) The control flow graph with conditional branching.

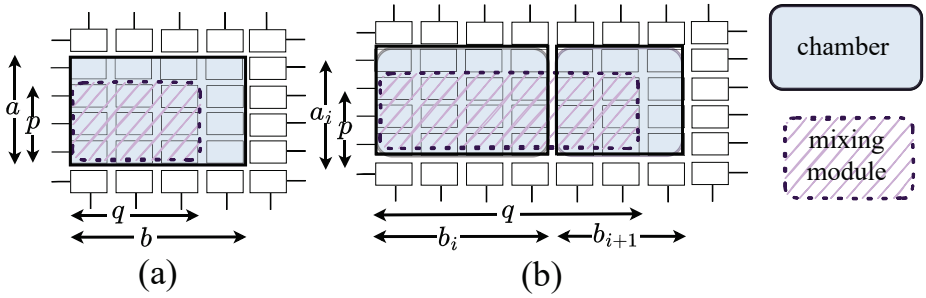


Fig. 7. (a) Internal fragmentation with homogeneous chamber, (b) external fragmentation with heterogeneous chambers.

If a mixer module (m) of size $p \times q$ is in ready queue ($\mathfrak{R}$) required to be placed on $\mathcal{K}$ available heterogeneous chambers $\{(a_1 \times b_1), (a_2 \times b_2), \ldots, (a_{\mathcal{K}} \times b_{\mathcal{K}})\}$. Then, external fragmentation can occur when the total available space is sufficient to place a mixer module ($p \times q$); however, no individual chamber is sufficient to accommodate it.

Table 1. Limitations of DMFB-Based *PBOS* [9] and Improvements in the Proposed Work

Aspect	*PBOS* [9]	Proposed Work
Mixing Model	Designed for only the simple $(k : k)$ mixing model, which mixes only two fluids of equal volumes.	Designed for flexible $(k_1 : k_2 : \cdots : k_n)$ mixing models, which mix multi-fluid (more than two) with unequal volumes.
Virtual Topology Design	Rigid, *chamber-based* topology limiting placement flexibility for using variable-sized mixers.	**Chamber-less, row-based** topology allowing adaptive placement of variable-sized mixers.
Fragmentation Handling	Suffers from internal fragmentation with homogeneous chambers and external fragmentation with heterogeneous chambers.	Eliminates internal fragmentation with chamber-less topology and handles external fragmentation using a **compaction** mechanism.
Assay Completion Time	Higher completion time due to chamber-based inefficient space utilization (e.g., 4 cycles for Figure 8(b–c)).	Reduced completion time due to efficient space utilization with chamber-less design and compact placement (e.g., 3 cycles for Figure 8(d)).
Reliability	No mechanism to produce reliable placement solutions, with focusing electrode degradation due to overuse.	Integrates a **reliability enhancer** to produce reliable placement solutions and ensure robust online execution.
Scalability and Suitability	Not scalable for larger and complex bioassays; inadequate to support the advanced operational flexibility offered by MEDA biochips.	Designed for MEDA biochips, scalable with **SAR-mixers abstraction** and supports **MEDA-specific operational flexibilities**.
Sample Preparation	Can only leverage primitive sample preparation algorithms, resulting in increased reagent use, additional mixing steps, and higher waste.	Compatible with advanced sample preparation algorithms that achieve the same reagent ratios with **fewer samples, fewer mixing steps, and minimal wastage**.

$$\begin{aligned} \iff & (\exists\, m \in \mathfrak{R} \;\mid\; \text{area}(m) = p \cdot q > 0) \\ & \wedge \left(\sum_{i=1}^{\mathcal{K}} a_i \cdot b_i \geq p \cdot q \right) \\ & \wedge \left(\nexists\, (a_i \times b_i) \;\mid\; a_i \cdot b_i \geq p \cdot q,\; i \in [1, \mathcal{K}] \right). \end{aligned} \tag{1}$$

In such scenarios, enough total free space is fragmented among smaller chambers, resulting in external fragmentation, as shown in Figure 7(b). The mixing graph shown in Figure 8(a), if required to place on the existing virtual topology [9] with **heterogeneous chambers**, then they suffer from internal fragmentation, as shown in Figure 8(c) at time $t = 2$.

In this article, we propose a row-based, chamber-less virtual topology where internal fragmentation cannot occur, and external fragmentation is handled by the procedure called compaction (as illustrated in LEMMA 3). Table 1 summarizes the key limitations of the DMFB-based *PBOS* [9] approach and highlights the improvements achieved in the proposed work. Unlike *PBOS*, which is restricted to simple two-fluid mixing, rigid chamber-based topology, and suffers from fragmentation and inefficient space utilization, our method introduces a chamber-less, row-based virtual topology that supports flexible multi-fluid mixing with unequal volumes. This design not only eliminates fragmentation through *compaction* but also significantly reduces assay completion time. In addition, the proposed framework integrates a reliability enhancer to mitigate electrode degradation, introduces the SAR-mixer abstraction to ensure the scalability of placement on MEDA biochips, and enables compatibility with advanced sample preparation algorithms that minimize reagent use and waste. Together, these advancements demonstrate the limitations of *PBOS* and establish the necessity of the proposed approach as a scalable, reliable, and resource-efficient online synthesis method tailored for MEDA biochips.

4 Motivation

The existing DMFB-specific virtual topology and *PBOS* [9] only supports mixing model $(k : k)$ to mix two fluids of equal volumes (shown in Figure 2(b)). However, MEDA biochips support more flexible mixing models, like multi-fluid mixing with unequal volumes $(k_1 : k_2 : \cdots : kn)$. If *PBOS* [9] considers homogeneous and heterogeneous chambers to synthesize MEDA-specific mixing graphs, then it suffers from internal and external fragmentation, respectively. For instance,

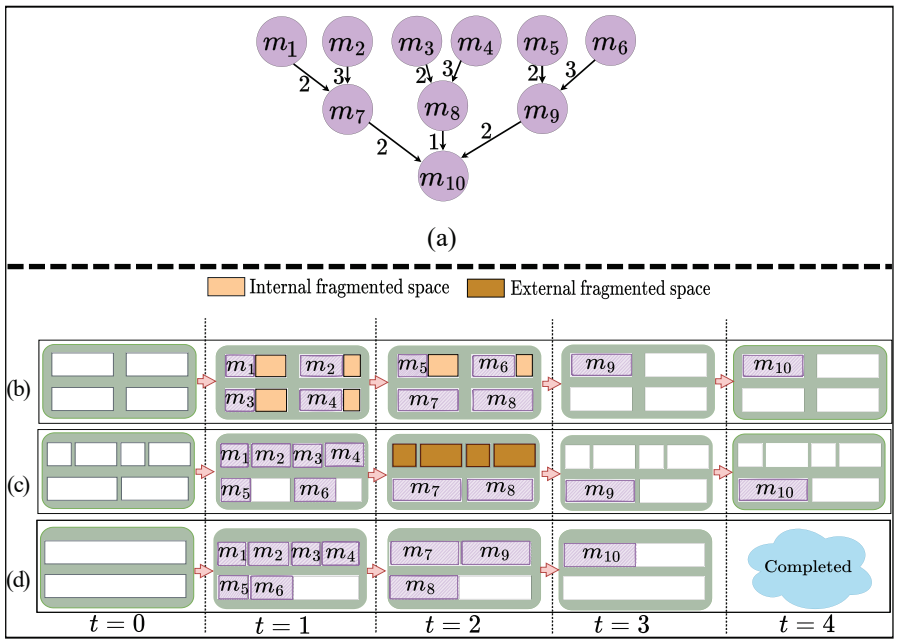


Fig. 8. Scheduling and placement of (a) an arbitrary mixing graph, on existing virtual topology [9] with (b) homogeneous chambers, (c) heterogeneous chambers, and (d) the proposed virtual topology with dynamic work chambers.

the MEDA specific mixing graph shown in Figure 8(a) [15, 16] suffers from internal and external fragmentation if synthesized with homogeneous and heterogeneous chambers, respectively, as shown in Figure 8(b) and Figure 8(c). As a result, the assay completion time in both scenarios increased to 4. Hence, *PBOS* [9] is not suitable for synthesizing complex mixing graphs designed for MEDA biochips. To overcome these drawbacks of the DMFB-based virtual topology with the existing online synthesis framework [9], we proposed a chamber-less, row-based virtual topology, and an online placer with a reliability enhancer for MEDA biochips, which reduced the assay completion time to 3 as shown in Figure 8(d). The proposed virtual topology is free from internal fragmentation. However, external fragmentation can occur when a placed module is positioned in the middle of a row, dividing the continuous free space into separate parts. To overcome this, we have proposed a *compaction* procedure that shifts all the placed modules to the left of the row, creating a continuous free space on the right side of the row, as shown in Figure 15(c-d). LEMMA 3 explains the *compaction* procedure in detail. The proposed virtual topology also overcomes the design gaps in sequential synthesis by handling placement failures, as shown in Figure 14, and eliminating the possibility of routing failure (Equation 3). Microelectrodes can be degraded due to the overuse of a particular region of the biochip, compromising the chip's reliability and leading to erroneous results. In the literature, except *WLSM* [5] and *RLPM* [6], no placement solution has considered the reliability of the chip as a concern. The occupied chip-area increases to enhance the reliability of the chip. For the mixing graph shown in Figure 9(a), the existing reliability-aware placements, (i) *WLSM* [5] incur higher area usage due to excessive distribution of electrode utilization and (ii) *RLPM* [6] suffers from higher algorithmic runtime to find the optimal solution using reinforcement learning as shown in Figures 9(b) and 9(c), respectively. This motivates us to develop an area-efficient online placer (*AOP*), which provides placement solutions with a lower

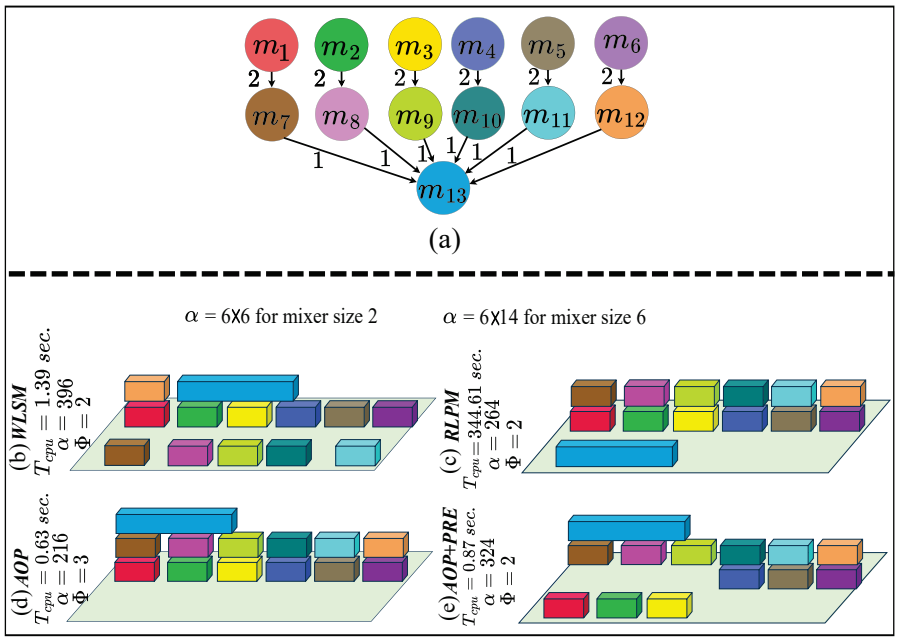


Fig. 9. Different placement results of (e) a mixing graph produced by (f) *WLSM* [5], (g) *RLPM* [6], and the proposed (h) *AOP*, and (i) *PRO+PRE*.

algorithmic runtime while minimizing occupied area, as shown in Figure 9(d). It also motivates us to develop a placement reliability enhancer (*PRE*) that uses placement solutions given by *AOP* to produce reliable placements with similar algorithmic runtime, as shown in Figure 9(e).

5 Problem Formulation

Before starting the placement, the fluidic operations in a DAG need to bind to the on-chip modules in M_{lib} and need to be scheduled (determining the start times of each fluidic operation without violating dependencies). Let $\mathcal{L}$ be the list containing the scheduling information of the operations, and $\mathcal{A}_{\mathcal{O}}$ be the placement sub-region to place the scheduled operations on-chip. The detailed problem formulation is presented as follows.

5.1 Area-Efficient Online Placement

In this work, we develop a virtual topology–based placement strategy in which the chip space is divided into placement sub-regions and routing sub-regions. The virtual topology modeling (described in Section 6.1) takes the entire chip space as input and returns the number of placement rows and routing channels with specified heights and widths. These sub-regions can be designed differently depending on the virtual topology modeling. However, once the virtual topology is finalized, it remains fixed during assay execution and can only be changed by redesigning the virtual topology.

Inputs: (i) **Scheduled List** $\mathcal{L} = \{v_i, Ts_i, Te_i\} \mid i \in [1, n]$, where Ts_i and Te_i are the start and end times of operation v_i of a mixing graph (DAG): $G(V = \{v_1, \cdots, v_n\}, E = \{e_1, \cdots, e_k\}) \mid e_p = \{(v_i, v_j), w_{i,j}\}; i, j \in [1, n]$ and $i \neq j$; $k \in [1, \frac{n(n-1)}{2}]$.

(ii) **Row-Place Sub-Region** $\mathcal{A}_{\mathcal{O}} = \{R_j, x_j, x'_j\} \mid R_j \in [1,\ N_p]$, where N_p is the number of placement rows, and x_j and x'_j represent the start and end points of the row R_j along X-axis, where the module can be placed in between.
Output: **Area-efficient Placement** $P_{\mathcal{O}} = \{v_i, R_j, ps\} \mid j \in [1,\ N_p]$ and $ps \in [x_j, x'_j]$, where v_i is placed on R_j (j^{th} *row*) at position *ps*.
Objective: The goal is to find an area-efficient placement of scheduled operations that minimizes algorithmic runtime (T_{cpu}), occupied chip area (α), and completion time (T_c). Since placement is an NP-hard optimization problem, we propose a scalable heuristic, the Area-efficient Online Placer (*AOP*), which produces sub-optimal solutions in minimal computational time, making it suitable for online synthesis.

5.2 Reliability-Aware Placement

Apart from area efficiency, the reliability of the biochip is also an important factor to consider during placement. Hence, we utilize the inputs of area-efficient placement and a reliability threshold (the maximum number of times an electrode can be actuated without degradation), based on the lifespan of the electrodes in the MEDA biochip, and formulate a reliability-aware placement as follows.
Inputs: (i) The **Area-efficient Placement Result ($P_{\mathcal{O}}$)**, (ii) **Overlapping threshold** Φ': Maximum number of modules that are allowed to use a specific microelectrode cell (x, y) for placement, where $\forall\ (x, y) \in \mathcal{A}_{\mathcal{O}}$.
Output: **Reliability-aware Placement** ($P'_{\mathcal{O}}$): A modified placement solution of the modules, such that the maximum overlapping value remains below the specified reliability threshold (Φ').
Objective: The objective is to determine a reliable placement of all modules under the overlapping threshold Φ' by invoking the Placement Reliability Enhancer (*PRE*). If certain modules cannot be placed reliably within the initial chip size, the recovery strategy is employed to compute the minimum number of additional placement rows required, ensuring that all modules are accommodated without violating Φ'.

Routing is the final step of the synthesis workflow, where the output of one module serves as the input for another module placed at different on-chip locations. Based on $\mathcal{L}$ and $P_{\mathcal{O}}$ or $P'_{\mathcal{O}}$, the job is to determine the **Routing Sub-task** $(\mathcal{R}_{sub}) = \{D_i, t, (x_s, y_s), (x_d, y_d)\}$, where D_i is the ith droplet, t is start time, (x_s, y_s) and (x_d, y_d) are the source and destination of the ith droplet, respectively. Finally produce the **Routing Result** $(\mathcal{R}) = \{D_i, pt\}$, where pt is path array $\mid pt = \{p_{[1,(x,y)]}, p_{[2,(x,y)]}, \cdots, p_{[t,(x,y)]}\}$, where $x, y \in \mathcal{A}_{\mathcal{R}}$ represents the position of D_i at time t.

6 Virtual Topology-Based Online Synthesis for MEDA Biochips

In this section, we will first discuss the modeling of the proposed virtual topology, followed by a discussion on the complete synthesis procedure based on this virtual topology.

6.1 Virtual Topology Modeling for MEDA Biochips

The virtual topology of DMFB is formally modeled as a functional partitioning of the physical electrode array into distinct operational regions. Instead of having distinct chambers for the fluidic operations, we have developed a row-based design where fluidic operations are dynamically placed inside the rows. The row height and SAR (Split and Recombine) mixer heights are kept equal. Therefore, the $2D$ placement problem within a row reduces to a $1D$ placement problem. This abstraction enables independent handling of module placement and droplet routing while ensuring physical realizability. The model uses the following notation:

6.1.1 Row-Based Virtual Topology Designing for MEDA. The proposed virtual topology groups microelectrodes into sub-regions based on assigned functions. The physical geometry of the chip:

Table 2. Notation for Virtual Topology Modeling

Variable	Description
$H \times W$	Dimensions of the electrode grid (cells)
$\mathcal{A}_{H\times W}$	Complete set of electrodes
$\mathcal{A}_{\mathcal{O}}$	Placement region for operational modules
$\mathcal{A}_{\mathcal{R}}$	Routing region for droplet transportation
N_p	Number of placement rows
h_p	Height of each placement row (cells)
w_p	Width of each placement row (cells)
h_r	Height of routing channel (cells)
w_r	Width of side routing channel (cells)

Given a grid $\mathcal{A}_{H\times W} = \{(y,x)|1 \le y \le H, 1 \le x \le W\}$,

$$\mathcal{A}_{H\times W} = \mathcal{A}_{\mathcal{O}} \cup \mathcal{A}_{\mathcal{R}}, \quad \mathcal{A}_{\mathcal{O}} \cap \mathcal{A}_{\mathcal{R}} = \emptyset. \tag{2}$$

The placement regions $\mathcal{A}_{\mathcal{O}}$ are defined as follows:

$$\mathcal{A}_{\mathcal{O}} = \bigcup_{i=1}^{N_p} \mathcal{A}_{\mathcal{O}}^{(i)}. \tag{3}$$

where each row $\mathcal{A}_{\mathcal{O}}^{(i)}$ is defined as

$$\mathcal{A}_{\mathcal{O}}^{(i)} = \left\{ (y,x) \middle| \begin{matrix} y \in \left[h_r + (i-1)(h_p+h_r) + 1, \quad h_r + (i-1)(h_p+h_r) + h_p\right], \\ x \in [w_r + 1, W - w_r] \end{matrix} \right\}. \tag{4}$$

The virtual topology partitions the $H\times W$ MEDA biochip into complementary placement ($\mathcal{A}_{\mathcal{O}}$) and routing ($\mathcal{A}_{\mathcal{R}}$) regions. The placement area comprises N_p parallel rows, each with fixed dimensions $h_p \times w_p$ where $w_p = W - 2w_r$. As formalized in Eqns. (2)–(4), row $\mathcal{A}_{\mathcal{O}}^{(i)}$ spans vertically from $y = h_r + (i-1)(h_p+h_r) + 1$ to $y = h_r + (i-1)(h_p+h_r) + h_p$. The first row starts at $y = h_r + 1$ and the subsequent rows begin after $h_p + h_r$ intervals. Each row has the fixed height h_p. The rows are horizontally bounded between $w_r + 1$ and $W - w_r$. Rows are inset by w_r from both edges of the chip. This model separates the placement region from the routing regions by explicit coordinate bounds.

Routing regions ($\mathcal{A}_{\mathcal{R}}$) are defined as follows:

(1) Top Horizontal Routing Path:

$$\mathcal{A}_{\mathcal{R}}^{\text{top}} = \{(y,x) \mid y \in [1, h_r],\ x \in [w_r + 1, W - w_r]\}. \tag{5}$$

(2) Inter-Row Routing Channels:
For each $i \in \{1, \ldots, N_p - 1\}$:

$$\mathcal{A}_{\mathcal{R}}^{\text{inter}}(i) = \left\{ (y,x) \middle| \begin{matrix} y \in [i(h_p+h_r) + 1,\ i(h_p+h_r) + h_r], \\ x \in [1, W] \end{matrix} \right\}. \tag{6}$$

(3) Bottom Horizontal Routing Path:

$$\mathcal{A}_{\mathcal{R}}^{\text{bot}} = \left\{ (y,x) \middle| \begin{matrix} y \in [H - h_r + 1, H], \\ x \in [w_r + 1, W - w_r] \end{matrix} \right\}. \tag{7}$$

(4) Side Routing Path:

$$\mathcal{A}_{\mathcal{R}}^{\text{sides}} = \{(y,x) \mid y \in [1, H],\ x \in [1, w_r] \cup [W - w_r + 1, W]\}. \tag{8}$$

The complete routing region is the union:

$$\mathcal{A}_{\mathcal{R}} = \mathcal{A}_{\mathcal{R}}^{\text{sides}} \cup \left(\bigcup_{i=1}^{N_p-1} \mathcal{A}_{\mathcal{R}}^{\text{inter}}(i) \right) \cup \mathcal{A}_{\mathcal{R}}^{\text{top}} \cup \mathcal{A}_{\mathcal{R}}^{\text{bot}}. \tag{9}$$

The routing region $\mathcal{A}_{\mathcal{R}}$ is systematically partitioned into four distinct sub-regions to enable efficient droplet transportation while avoiding interference with placement areas. The top and bottom horizontal paths ($\mathcal{A}_{\mathcal{R}}^{\text{top}}$ and $\mathcal{A}_{\mathcal{R}}^{\text{bot}}$) provide dedicated routing space above the first and below the last placement row respectively, each spanning the full row width ($w_r + 1$ to $W - w_r$) with fixed height h_r. Between consecutive placement rows, inter-row channels ($\mathcal{A}_{\mathcal{R}}^{\text{inter}}(i)$) create horizontal routing corridors of height h_r, vertically aligned from $y = i(h_p + h_r) + 1$ to $i(h_p + h_r) + h_r$ across the entire chip width. Vertical movement is facilitated by side channels ($\mathcal{A}_{\mathcal{R}}^{\text{sides}}$) running along both edges of the chip. The union (Equation 9) of these components ensures conflict-free droplet routing with placement region while maintaining minimum 4×4 cell routing paths for unit-volume droplets.

Virtual topology design constraints:

(1) Height Constraint:

$$N_p \times h_p + h_r(N_p + 1) \leq H. \tag{10}$$

— Ensures sufficient space for N_p rows and $N_p + 1$ routing channels
— Violation leads to overlapping regions

(2) Width Constraint:

$$2 \times w_r + w_p \leq W. \tag{11}$$

— Determine row width w_p after reserving side routing channels
— w_r must accommodate droplet size and electrode activation patterns

(3) Non-Negativity Constraints:

$$h_p \geq h_{p,\min}, \quad w_r \geq w_{r,\min}, \quad N_p \geq 1. \tag{12}$$

— $h_{p,\min} = 6$ ensures mixer modules with interference region fits vertically
— $w_{r,\min} = 4$ meets the minimum droplet routing requirements while not considering droplet shape and size morphing

(4) Region Disjointedness and Complete Coverage:

$$\mathcal{A}_{H \times W} = \mathcal{A}_{\mathcal{O}} \cup \mathcal{A}_{\mathcal{R}}, \quad \mathcal{A}_{\mathcal{O}} \cap \mathcal{A}_{\mathcal{R}} = \emptyset.$$

— Critical for avoiding placement and routing regions overlapping
— Critical for complete chip coverage

The virtual topology design is governed by four fundamental constraints that ensure physical realizability and functional correctness. First, the **height constraint** (Equation 10) guarantees sufficient vertical space for all placement rows and their associated routing channels, where N_p rows of height h_p require $N_p + 1$ routing channels of height h_r (accounting for top, bottom, and inter-row paths). Second, the **width constraint** (Equation 11) maintains minimum row width w_p by reserving w_r cells on both edges for side routing channels. Third, **non-negativity constraints** (Equation 12) enforce minimum dimensions: $h_p \geq 6$ accommodates mixer modules with interference region (guard ring), $w_r \geq 4$ ensures routing without droplet shape and size morphing, and $N_p \geq 1$ guarantees at least one placement row. Finally, the **disjointness property** ensures strict separation ($\mathcal{A}_{\mathcal{O}} \cap \mathcal{A}_{\mathcal{R}} = \emptyset$) while maintaining complete coverage ($\mathcal{A}_{H \times W} = \mathcal{A}_{\mathcal{O}} \cup \mathcal{A}_{\mathcal{R}}$) of all electrodes. Together, these constraints prevent overlapping regions while meeting the physical requirements of MEDA biochip operations.

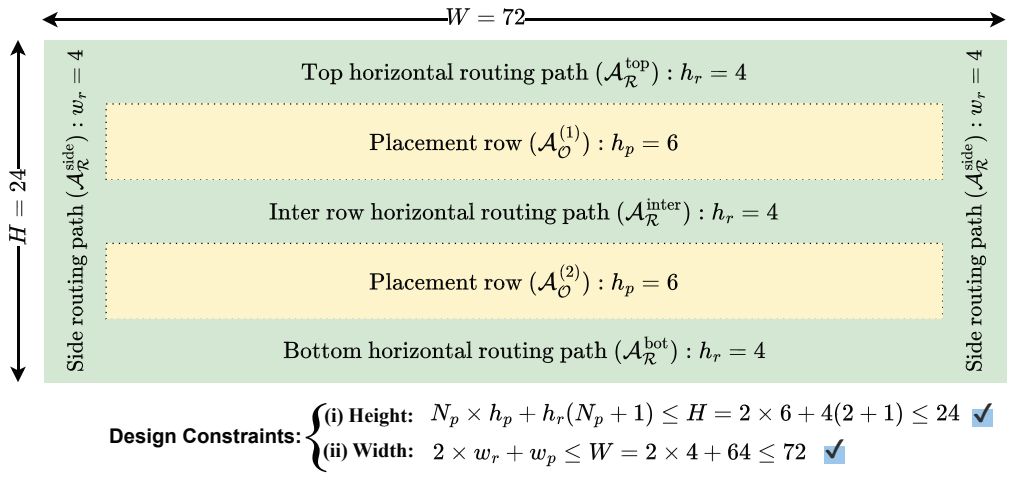


Fig. 10. Virtual topology designing for a 24 × 72 MEDA biochip with different sub-regions.

Figure 10 illustrates the proposed row-based virtual topology design of a MEDA biochip of size 24 × 72. It involves logically dividing the entire chip area into distinct functional regions: (i) routing regions ($\mathcal{A}_{\mathcal{R}}$) and (ii) placement regions ($\mathcal{A}_{\mathcal{O}}$) with its corresponding sub-regions like (top horizontal routing path ($\mathcal{A}_{\mathcal{R}}^{\text{top}}$), inter routing paths ($\bigcup_{i=1}^{N_p-1} \mathcal{A}_{\mathcal{R}}^{\text{inter}}(i)$), bottom routing paths ($\mathcal{A}_{\mathcal{R}}^{\text{bot}}$), side routing paths ($\mathcal{A}_{\mathcal{R}}^{\text{sides}}$), placement regions with rows ($\bigcup_{i=1}^{N_p} \mathcal{A}_{\mathcal{O}}^{(i)}$)). Including the interference region (guard ring), we have considered that the height of the mixing modules will be $(4 + 2) = 6$ and the width varies to accommodate different sizes of the mixers. Hence, the height of the row needed to be at least 6. In this work, we have not considered droplet morphing to route droplets with different sizes and shapes. Hence, we kept the $h_r = 4$ for horizontal routing paths and $w_r = 4$ for side routing paths, which is sufficient to route the unit volume of droplets with the stalling-based collision avoidance mechanism [19].

6.1.2 Mixer Configuration and Deadlock-Free Reagent Loading. The chamber-based virtual topology [9] suffers from fragmentation issues in synthesizing mixing graphs with variable-sized mixers, as depicted in LEMMA 1 and LEMMA 2. In this work, we propose a chamber-less virtual topology consisting of rows (placement areas) without predefined chambers. All the fluidic operations (excluding routing) are performed inside the rows. Here, the height of the mixing modules remains constant, and the width increases to accommodate larger mixers. By maintaining the same height for the mixer modules as well as for the rows, the $2D$ placement problem is reduced to a $1D$ placement problem. This advantage can be leveraged by employing a faster MEDA-supported mixing model, known as **split-and-recombine** (**SAR**) mixing. Figure 11(a) shows a 2 size SAR mixer with the required steps where each droplet occupied 4 microelectrodes. If the output volume of the mixer is b, then the dimension will be $4 \times ((2 \times b) + 1)$, excluding the interference region (guard ring). In Figure 11(b), the extra +1 is a separator between the input and output fluids. It is required when a mixing operation is completed, but the output fluid has not been transported yet, while the input fluid needed for the next timestamp is already loaded into the mixer. If the entry and exit ports of a chamber are the same, it can suffer from routing deadlock [9]. Different entry and exit ports are introduced to prevent routing deadlocks. The input fluid is loaded from entry ports, and the output fluid exits through the exit ports, as shown in Figure 11(b).

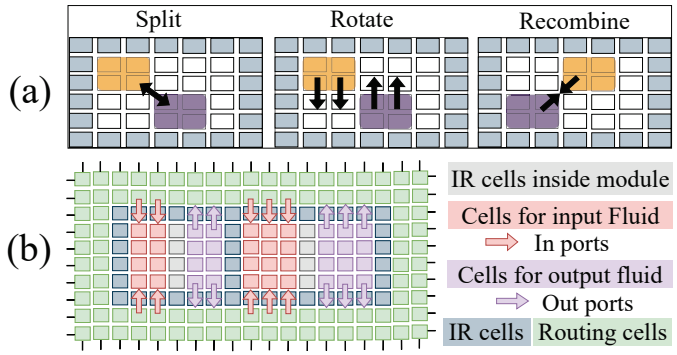


Fig. 11. (a) SAR mixing steps, and (b) chamber-less virtual topology design on MEDA.

ALGORITHM 1: *List_Scheduling(G, B)*

```
Compute priority p(v), ∀v ∈ V, based on distance from sink node;
Initialize ready queue ℜ ← {v ∈ V | ∀u, (u, v) ∉ E};
Initialize resources list R_m = {m_1, m_2, ··· , m_n} |(v_i, m_i) ∈ B;
Initialize ℒ = {};
while ℜ ≠ NULL do
    Select highest priority task v from ℜ;
    Assign v to resource m_i ∈ R_m with times Ts_t, Te_t;
    Add (v, Ts_t, Te_t) to ℒ;
    Update R_m and remove v from ℜ;
    for each dependent task u do
        Remove dependency (v, u);
        if u has no incoming edges then
            Add u to ℜ;
```

6.2 Online Synthesis for MEDA Biochips

Here, we discuss the complete online synthesis framework, including the proposed *AOP* and *PRE* for MEDA biochips. It integrates a scheduler [3], and a router [19] with the proposed placer and reliability enhancer. The proposed work can be integrated with any scheduler and router to develop a complete online synthesis framework. However, we have used the scheduler [3] and router [19] to facilitate a fair comparison with the existing online synthesis framework [9]. The working flow of the proposed *FOS* framework is illustrated in Figure 12(a).

6.2.1 Module Binding and Operation Scheduling. This study employs a straightforward binding approach, where each operation is exclusively paired with a specific module in M_{lib}. We utilize list scheduling (*LS*)[3], which identifies ready nodes at each timestamp and prioritizes them based on their proximity to the sink node [3]. In *LS*, the ready list dynamically manages tasks ready for execution once all dependencies are resolved. Initially, tasks without incoming edges (indicating no dependencies) are added to the ready list. When a task ends, new tasks with resolved dependencies are continuously added, ensuring a steady flow of executable tasks. *LS* uses heuristic approaches to efficiently prioritize tasks and update the ready list to produce scheduling results with minimized

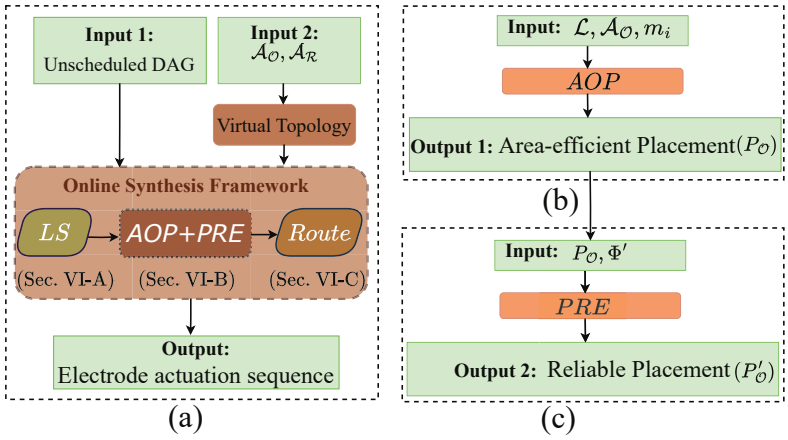


Fig. 12. Flowchart of the proposed (a) *FOS*, (b) *AOP* and (c) *PRE*.

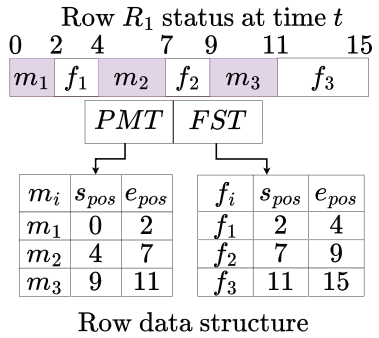


Fig. 13. Row data structure to manage the placed module and free space information.

T_l, reducing computational complexity and runtime. Instead of *LS*, other heuristic-based scheduling methods, such as [18], align with our proposed virtual topology and online synthesis framework. However, *LS* is used for simple and faster assay execution. Figure 4(c) and 4(e) show the binding and scheduling, respectively. The output scheduled mixing graph is used as input for the placer. In **Algorithm 1**, line no. 1 to 4, compute the initial priorities, initialize the ready queue, available resources, and an empty schedule list. The main loop starts at line no. 5 and continue until all the operations are scheduled. Line no. 6-8, identify the highest-priority node and schedule it; then, from line no. 9 recalculate the resources and update the ready queue.

6.2.2 Area-Efficient Online Placer with Reliability Enhancer. On top of the proposed virtual topology defined in Section 6.1, an area-efficient online placer *AOP* and a placement reliability enhancer *PRE* are presented, where operation modules are confined to specific rows ($R_j \in \mathcal{A}_{\mathcal{O}}$). The working flow of *AOP* and *PRE* is shown in Figure 12(b) and 12(c), which includes (i) determining the best area fit placement position inside rows, (ii) removing modules from the row once operations are completed, (iii) *compaction* to manage external fragmentation, and (iv) invoking reliability enhancer if placement results violate the overlapping threshold (Φ'). The row-data structure, shown in Figure 13, manages the status of each row. It is organized as a two-layered structure: the first layer

maintains pointers to individual rows, while the second layer consists of two 2D tables—the **Placed Module Table** (**PMT**) and the **Free Space Table** (**FST**). The PMT and FST track row allocation and internal placement within rows, thereby enhancing placement efficiency. Each module m_i and free space f_i is represented by its start and end positions, s_{pos} and e_{pos}, as illustrated in the Figure 13.

Algorithm 2 takes the Scheduled List ($\mathcal{L}$), and the row-place sub-region ($\mathcal{A}_{\mathcal{O}}$) as inputs. It produces the placement result ($P_{\mathcal{O}}$) with minimized occupied area (α) and assay completion time (T_c). The main loop begins at line no. 1 and continues until all the operations are placed. Line no. 2 initialize the solution array. First, at line no. 3, the scheduled list is sorted according to the start time of the scheduled operations. Line no. $4-5$ pop the scheduled operations one by one and determine the size of the corresponding module. Line no. 6-9 check each row (R_j) for available free space. If sufficient space is found to place m_i, then recorded in *sol* at line no. 10. If no free space is available, line no. 12-14 checks if external fragmentation is the issue. If the sum of total fragmented spaces can accommodate m_i, then at line no. $15-16$ *Compaction*(R_j) is invoked to address the external fragmentation of R_j and record it into the *sol* array. Following *Compaction* (if needed), *AOP* selects the best area fit solution from the *sol* array at line no. 16 and provide placement results that minimize the occupied area, without prioritizing chip reliability. If the *sol* array is not null,

ALGORITHM 2: $AOP(\mathcal{L}, \mathcal{A}_{\mathcal{O}})$

```
1  while L ≠ NULL do
2      sol = []; // Store valid placement solutions
3      Sort L in ascending order of the scheduled start time (Ts);
4      m_i ← POP(v_i from L); // Pop the earliest scheduled operation
5      b ← Size of module m_i;
6      for j ← 1 to N_p do
7          f ← R_j.FST; // Free space table of j^th row
8          for k ← 1 to len(f) do
9              if b ≤ (f[k][2] − f[k][1]) then
10                 sol.append(f[k]);
11                 break;
12             if k == len(f) then
13                 Frag ← Total Fragmented Free Space;
14                 if Frag ≥ b then
15                     Compaction(R_j);
16                     sol.append(f[1]);
17     if sol ≠ NULL then
18         P_O ← BestAreaFit(sol, m_i); // Area-efficient placement
19     else
20         PUSH(m_i into L); // No valid placement found, loop until already
               placed modules are completed
21         Find out all the reachable nodes p = {p_1, p_2, ··· , p_n} from m_i;
22         for i ← 1 to n do
23             Ts_i = Ts_i + 1;
```

ALGORITHM 3: *BestAreaFit(sol, m_i)*

```
1 i ← 1;
2 while i ≤ len(sol) do
3     if (sol[1].epo − sol[1].spo) ≥ (sol[i].epo − sol[i].spo) then
4         swap(sol[1], sol[i]);
5     i ← i + 1;
6 return sol[1];
```

then at line no. 18 the *BestAreaFit* function is invoked. The *BestAreaFit* function takes the *sol* array as input, which contains all valid placement options for a module m_i. Among these, the function selects the placement that leaves the least possible fragmentation. In other words, it chooses the smallest available space that can accommodate m_i. Figure 15(a) illustrates the available placement options for m_4, while Figure 15(b) shows the final best-fit placement. **Algorithm 3** outlines the exact steps of the Best fit policy. In **Algorithm 3**, line no. 2–6 search through the *sol* array to identify the placement with the smallest available space for m_i.

Overcome Design Gaps in Sequential Synthesis: However, if the *sol* array is null, then there is no adequate space in the chip to place m_i. It shows the placement failure for the given schedule. As discussed in Section 2-B, the placement and routing failure issues in sequential synthesis resulted in design gaps. Line no. 19–23 show a resource (chip size) constraint rescheduling strategy to get a valid placement result. Hence, until the already placed operations are finished, m_i cannot be placed. Therefore, at line no. 20, m_i again pushed back into $\mathcal{L}$ and tried to get placed at the next timestamp. From line no. $21-23$, it identifies all nodes reachable from m_i using **Depth-First Search** (**DFS**) and shifts them to the next timestamp. Rescheduling a single node has a runtime complexity of $\mathcal{O}(V+E)$. In the worst-case scenario, all nodes may need to be rescheduled, which requires $\mathcal{O}(V \times (V+E))$ time. If a node is in the critical path and needs to be rescheduled to the next timestamp, then the completion time increases.

Rescheduling a critical-path node to the next timestamp: Suppose a module of size $p \times q$, corresponding to a node on the **critical path**, must be placed into one of the available free rectangles $\{a_i \times b_i\}_{i=1}^{n}$. Splitting across multiple free regions is **not allowed**, but the module may be **rotated by** 90°, i.e., its orientation can be either $p \times q$ or $q \times p$.

Area constraint: A necessary condition for placement of a module with $p \times q$ dimension is

$$\sum_{i=1}^{n} a_i b_i \;\geq\; pq.$$

However, this condition alone is not sufficient, since an area must also be available in the correct geometric form. The following condition leads to the infeasibility of the placement, even when the necessary condition is met

$$\forall i \in \{1, \ldots, n\}, \quad \big[(a_i < p \vee b_i < q) \;\wedge\; (a_i < q \vee b_i < p)\big].$$

Feasibility condition for placement: The node can be placed at the current timestamp **if and only if** there exists a free rectangular space that can accommodate the module in some orientation:

$$\exists i \in \{1, \ldots, n\} \quad \big[(a_i \geq p \wedge b_i \geq q) \;\vee\; (a_i \geq q \wedge b_i \geq p)\big].$$

Impact of rescheduling the critical path node to the next timestamp: If no free rectangle satisfies the feasibility condition, then the module cannot be placed at the current timestamp. Since the module belongs to the critical path, it must be rescheduled to the **next timestamp**, which directly increases the **overall completion time** of the bioassay.

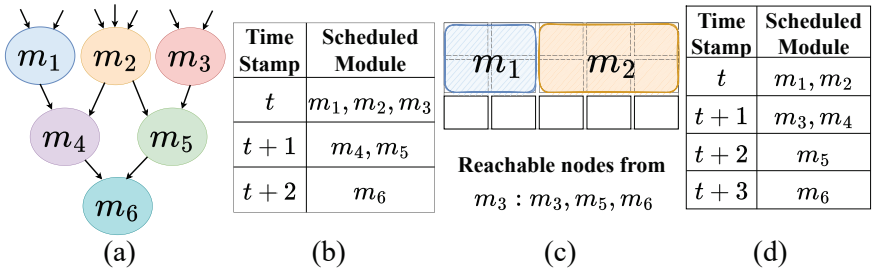


Fig. 14. For (a) a mixing graph, (b) initial scheduling for a 5 × 3 MEDA chip, (c) placement failure at time t, and (d) revised schedule.

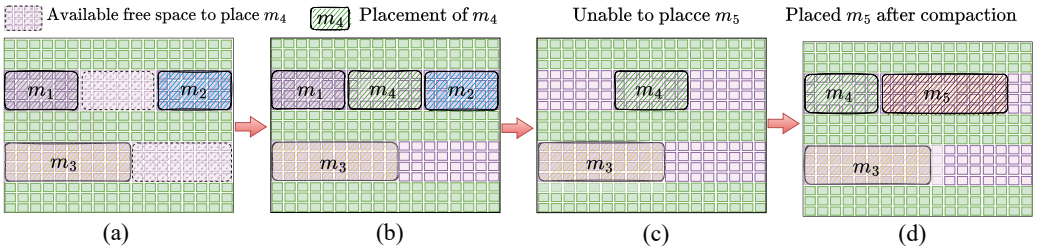


Fig. 15. (a) Possible placement solutions for m_4, (b) best-fit placement for m_4, (c) insufficient space to place m_5 before compaction, (d) placement of m_5 after compaction.

Illustrative example: Figure 14(c) shows the placement failure at time t of a mixing graph with initial scheduling shown in Figure 14(b), of a mixing graph (Figure 14(a)), where 5 cells are free; however, it cannot accommodate m_3 (required 4 free cells only). Since the reachable nodes from m_3 are m_3, m_5, m_6, to get a valid solution, all these nodes are shifted to the next timestamp, and the current schedule is updated, shown in Figure 14(d). Here, shifting the critical path node m_3 to the next timestamp resulted in an increase in the completion time. By doing this dynamically, we update the existing schedule and overcome placement failures in sequential synthesis without rescheduling the entire mixing graph every time a placement failure occurs.

Routing failures in sequential synthesis can be avoided using the proposed virtual topology design constraint Equation 3. This constraint dedicates each microelectrode to either module placement or droplet routing, ensuring that placed modules do not interfere with routing paths.

Placing a module on an $N \times N$ grid takes $\mathcal{O}(N^2)$. However, the proposed virtual topology divides the entire space into N_p rows, where $N_p << N$. The height of a row is fixed, while the width is equal to the width of the chip. If the *FST* is sorted based on the starting position along the X-axis, placement of a module in a row will take $\mathcal{O}(\log N)$ time. This process will take $\mathcal{O}(N_p \cdot \log N)$ time for N_p rows, which makes it faster and more suitable for online synthesis. Finding reachable nodes to overcome the design gaps in sequential algorithms takes $\mathcal{O}(V + E)$, where V and E denote the number of nodes and edges in the mixing graph, respectively. The *BestAreaFit* function finds the best area fit placement for m_i among N_p Rows. Figure 15(a) shows the available locations to place m_i, and Figure 15(b) shows the best area fit placement for m_i. All solutions are valid inside *sol*, and the *BestAreaFit* function returns the smallest among them. In the worst case, each row can provide a valid placement. Hence, the time complexity of *BestAreaFit* is $\mathcal{O}(N_p)$.

The proposed virtual topology has no internal fragmentation issue. However, external fragmentation can occur due to previously placed modules, as shown in Figure 15(c). *Compaction* is invoked

ALGORITHM 4: *Compaction*(R_j)

1 $\mathcal{P} \leftarrow R_j.PMT$; // Placed module table of jth row
2 $S_f \leftarrow Sort(R_j.FST)$ according to starting position;
3 **for** $i \leftarrow 1$ **to** $len(\mathcal{P})$ **do**
4 **for** $k \leftarrow 1$ **to** $len(S_f)$ **do**
5 **if** $\mathcal{P}[i][s_{pos}] \geq S_f[k][s_{pos}]$ **then**
6 $d \leftarrow \mathcal{P}[i][s_{pos}] - S_f[k][s_{pos}]$;
7 $Move.Left(\mathcal{P}[i], d)$;
8 $Merge(S_f[k], S_f[k+1])$; // Merge fragmented spaces
9 $Update(R_j.PMT)$;
10 $Update(R_j.FST)$;

when a row cannot place a module due to external fragmentation. It shifts all the placed modules on the left side of the row to create a single continuous free space on the right. Figure 15(c) shows the chip status with insufficient space to place m_j, and Figure 15(d) shows the placement after *Compaction*. In **Algorithm 4**, line no. 1-2 take *FST* and *PMT* as inputs for R_j. Line no. 3 iterates through the already placed modules in row R_j. Line no. 4-5 identify if any placed modules are to the right of the free space. If this is true, the distance d is calculated at line no. 6, and the placed module is moved left by distance d at line no. 7. At line no. 8, the current free space is merged with the consecutive free spaces to the right, and *PMT* and *FST* are updated at line no. 9-10. For *Compaction*, we must traverse R_j, which takes $\mathcal{O}(N)$ time for an $N \times N$ grid.

Lemma 3. *External fragmentation is removed by "Compaction" in chamber-less virtual topology.*

The chamber-less virtual topology divides the whole chip into multiple placement areas and routing channels, shown in Figures 11(b) and 8(d). Each of the placement areas is identified as a row of cells of height h_p and width w_p and defined as (R_j, x_j, x'_j), where R_j is the ID for the jth row. The starting and ending X-coordinates of the row are represented as x_j, x'_j, respectively. The modules to be placed on these rows are SAR mixers, which allowed us to fix their heights with varying widths. We have considered the heights (p) of all the modules to be the same as the height of a row, i.e., $p = h_p$, while their widths q can vary to accommodate different sizes of the SAR mixers. Suppose the mixer m_i is placed on j^{th} row R_j, which started from x_i along the X-axis and $x_j \leq x_i \leq x'_j$, the fragmented space is divided into two, (i) $p \times (x_i - x_j)$ and (ii) $p \times (x'_j - (x_i - 1 + q))$. After *compaction*, it will become a continuous free space of size $p \times (x'_j - q)$. If $q \geq (x_i - x_j) \wedge q \geq x'_j - (x_i + q)$, but $q \leq x'_j - (x_j + q)$, then m_i can be placed only after *compaction*. For multiple fragmented spaces, the same procedure needs to be repeated.

For example, the module m_5 cannot be placed due to a fragmentation issue, where only a single module m_4 was placed in row R_1 as shown in Figure 15 (c). The starting position (x_j) and ending position (x'_j) of row R_1 are 1 and 18, respectively, with $h_p = p = 6$. The starting position (x_i) of the already placed module m_4 is 7, having a width (q) = 6. Now the total free space in R_1 is fragmented into two (i) $p \times (x_i - x_j) = 6 \times (7 - 1) = 36$ and (ii) $p \times (x'_j - (x_i - 1 + q)) = 6 \times (18 - (7 - 1 + 6)) = 36$, where $6 \times 10 = 60$ required to place m_5. By shifting all the free space to the right side of the row, *compaction* creates a continuous free space of $p \times (x'_j - q) = 6 \times (18 - 6) = 72$, which is able to place m_5 in R_1 as shown in Figure 15 (d).

Obstacle-avoiding parallel droplet routing on a $2D$ grid is an NP-hard problem, as it involves finding the shortest path while avoiding both static and dynamic obstacles. In contrast, the shifting operation used for *Compaction* is much simpler. Shifting refers to moving all modules in a row uniformly toward the left boundary, thereby creating a continuous free space on the right side. This process does not require obstacle avoidance since all the modules shift in the same direction. In the worst case, a droplet may need to move from the rightmost to the leftmost position of the chip. The time complexity of this operation is $\mathcal{O}(W)$, where W is the chip width. Thus, shifting can be executed in polynomial time with linear complexity, and its computational cost is negligible compared with NP-hard routing. Moreover, while each single-step droplet movement takes the same amount of time in both routing and shifting, routing requires additional NP-hard path finding, whereas shifting does not. This makes the computational overhead of shifting negligible compared with the routing overhead. *Compaction* also incurs a small overhead in assay completion time. Table 5 shows a comparison of the average number of routing sub-tasks, routing length, routing time, average number of times compaction is invoked, compaction length, and time required for performing compaction. The results demonstrate that the overhead introduced by compaction is negligible when compared with routing. Thus, the time required for compaction has an insignificant impact on the overall assay completion time. It is essential to note that the assay completion time is primarily determined by operational time (e.g., mixing, dispensing) and routing time, with compaction contributing only marginally to the total.

Algorithm 5 determines a reliable placement solution from the result of *AOP*. Line no. 1 calculates the overlapping value Φ where $\sigma_{(x,y)}$ indicates the overlapping frequency of a cell (x, y) that is used the maximum number of times for module placement. Initialize **Unreliable_Module** list at line no. 3 to keep track of the modules that cannot be placed reliably within the given chip size. Then, it checks if the overlapping value exceeds the threshold at the line no 4. Then *PRE* identifies all modules where cell (x, y) violates the overlapping threshold at line no. $5-7$. Line no. $8-10$ determine the possible alternative placement locations for those modules. At line no. 11, it chooses an alternative placement location (if found) that does not cause Φ' violation. At line no. $12-14$, it replace the modules on the new location and updates $P_{\mathcal{O}}$. However, if the alternative placement location is not found at line no. 12, then it is recorded in **Unreliable_Module** at line no. 16. At line no. 17 if **Unreliable_Module** is Null, that means all the modules are placed reliably on the given chip. If **Unreliable_Module** is not Null, that means modules in **Unreliable_Module** cannot be placed reliably on the given chip. In Section 7.3 (Figure 19), we observe that the overlapping threshold (ϕ') increases as the chip area (α) decreases. Since higher placement reliability is achieved with smaller values of ϕ', reliable placement of all modules requires a larger chip size. Our approach utilizes the proposed row-based virtual topology for placing mixer modules, where the height of the modules is fixed and the width varies according to their size. To guarantee reliable placement, the recovery strategy determines the minimum number of additional rows required to accommodate those modules that could not be placed reliably in the initial layout. The recovery strategy (Line no. 20–30) is invoked when certain modules cannot be relocated reliably and are collected in the set **Unreliable_Module**. An event queue is initialized at line no. 20. Each unreliable module is first transformed into two events: an allocation event at its start time and a release event at its end time (Line no. 21–23). These events are sorted chronologically, with releases processed before allocations to ensure correctness at line no. 24. A sweep-line scan is then performed (Line no. 26–28) to track the total required width over time. The maximum concurrent width encountered during this scan is stored in $width_{(exten)}$. Finally, the required number of additional placement rows is computed as $\left\lceil width_{(exten)}/w_p \right\rceil$ at line no. 29, where w_p is the width of a placement row and N_p is the number of placement rows. Since each module contributes exactly two events, the overall time complexity

ALGORITHM 5: $PRE(\Phi', P_{\mathcal{O}})$

$\Phi = max(\sigma_{x,y},\ \forall(x,y) \in \mathcal{A}_{\mathcal{O}})$;
$P'_{\mathcal{O}} \leftarrow P_{\mathcal{O}}$;
Unreliable_Module = [];
if $\Phi > \Phi'$ **then**
  for *each cell*$(x,y) \in \mathcal{A}_{\mathcal{O}}$ **do**
    if $\sigma_{x,y} \geq \Phi'$ **then**
      Identify all modules used cell (x,y) for placement;
      for *each module* m_i **do**
        Find alternative placement locations for m_i on the same timestamps;
        $Loc = \{Loc_1, Loc_2, \cdots Loc_n\}$;
        Choose Loc_i where $max(\sigma_{x,y}) \leq \Phi',\ \forall x, y \in Loc_i$;
        if *Such* Loc_i *exists* **then**
          Place m_i at Loc_i;
          Update $P'_{\mathcal{O}}$;
        else
          Unreliable_Module$.append([Ts_i, Te_i, width(m_i)])$;
if Unreliable_Module== *NULL* **then**
  Reliable placement for all the modules;
else
  `// Recovery strategy`
  $Events \leftarrow []$;
  for *each* $[Ts_i, Te_i, width(m_i)] \in$ **Unreliable_Module do**
    $Events.append(Ts_i, +width(m_i))$;
    $Events.append(Te_i, -width(m_i))$;
  Sort *Events* by time;
  $current \leftarrow 0$; $width_{(exten)} \leftarrow 0$;
  for *each* (t, δ) *in Events* **do**
    $current \leftarrow current + \delta$;
    $width_{(exten)} \leftarrow \max(width_{(exten)}, current)$;
  $N_p = N_p + \left\lceil width_{(exten)}/w_p \right\rceil$;// where N_p and w_p are the number and width of placement rows.
  Repeat the same procedure with N_p (updated) number of rows;

of the recovery procedure is $O(U \log U)$, where $U = |\textbf{Unreliable_Module}|$, dominated by the event-sorting step. This approach ensures reliable placement of all modules by repeating the same procedure with the updated number of placement rows. Since the proposed virtual topology fixes the module height, the recovery strategy determines the extra width requirement and translates it into the necessary number of additional rows. In this way, the placement remains feasible and consistent under reliability constraints.

Illustrative Example: Figure 16 illustrates the recovery procedure for three unreliable modules m_1, m_2, and m_3, with volumes of 4, 6, and 4, corresponding to derived widths of 9, 13, and 9,

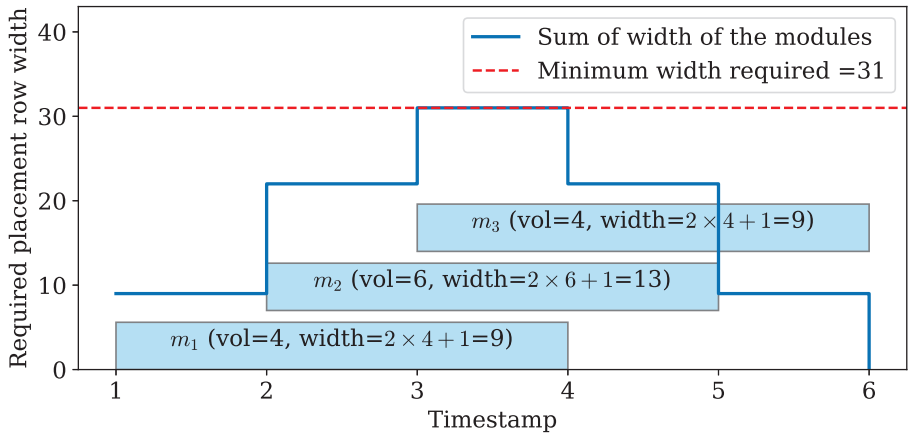


Fig. 16. Illustration of the recovery strategy with three unreliable modules. The sweep-line scan determines the maximum concurrent width $width_{(exten)} = 31$, which is then translated into the required number of additional placement rows.

ALGORITHM 6: Routing($\mathcal{R}_{sub}$)

```
1  Initialize droplets set D = {D_1, D_2, ..., D_n};
2  Sort D by descending Manhattan distance;
3  for each droplet D_i ∈ D do
4      Compute shortest path P_i for D_i from (x_s, y_s) to (x_d, y_d);
5      for j = 1 to i − 1 do
6          if P_i intersects with other droplet path P_j then
7              if P_j is longer than P_i then
8                  Stall D_i until D_j clears the path;
9          Move D_i along P_i;
10         Update D_i position to (x_d, y_d);
11         Remove D_i from D;
```

respectively. Over the timeline, the occupied width varies as modules enter and leave the chip. The peak occurs when all three modules overlap simultaneously, yielding a maximum concurrent width of $width_{(exten)} = 31$. To accommodate this demand, the chip must be extended by $\lceil width_{(exten)}/w_p \rceil$ placement rows, where w_p denotes the placement row width.

6.2.3 Droplet Routing. It is the final step in the synthesis flow. A modified version of [19] has been adopted in this work for fast routing. It operates in two phases: (i) Path Finding: where obstacle-avoiding routing paths for each routing sub-task in $\mathcal{R}_{sub}$ with droplet sources and destinations are determined using a simplified maze routing method, and (ii) Compaction for Parallel Routing: after individual routing paths are found, compaction is performed to enable simultaneous droplet routing without collision to provide the routing result $\mathcal{R}$. Unwanted collisions are avoided by prioritizing the longer path. **Algorithm 6** starts by initializing the set of droplets D to be transported, calculating Manhattan distances between sources and destinations, and sorting them by distance at line no.

1-2. At line no. 4-7, it computes the shortest path for each droplet and checks whether it intersects with the path of other droplets. If a collision is detected, the droplet with the longest path proceeds, while the shorter path droplet is stalled at line no. 8. In line no. Between 9 and 11, the droplets move along their designated paths and are removed from set D upon reaching their destinations. This process continues from line no. 3 for other droplets until set D is empty. This approach ensures efficient and rapid routing while preventing unwanted collisions. Any other routing algorithm for MEDA biochips can be easily integrated with the proposed placer and reliability enhancer. However, the routing algorithm [19] is used for the sake of comparison with existing online synthesis [9], and also provides a faster solution with lesser complexity.

7 Simulation Results

We implement all the state-of-the-art methods, namely *PBOS* [9], *EOPS* [23], *WLSM* [5], *RLPM* [6], and the proposed methods *AOP*, *PRE* and *FOS* using Python programming language. All simulations were conducted on a computer with an octa-core Intel Xeon(R) processor and 32GB RAM, running the 64-bit Ubuntu operating system. We have generated several mixing graphs using a state-of-the-art sample preparation method for MEDA biochips, namely *STWM* [16], and used those as the testcases to study the comparative performances in the following sub-sections.

7.1 Performance of *FOS* over MEDA-Tuned *EOPS* [23] and *PBOS* [9]

The assay completion time increases with the increase in height (h) of the mixing graph. Hence, by varying h from 3 to 7, we consider 50 testcases for each height (total 250 testcases grouped as $\mathcal{G}_i$s as shown in Table 3), all are generated using *STWM* [16] sample preparation method. The average assay completion time (T_c) and the average number of routing sub-problems (ρ), which indicates the number of routing stages between mix operations, are compared among the offline approach *EOPS* [23] as a benchmark, *PBOS* [9], both tuned for MEDA, and the proposed *FOS*. It is observed that with increasing h, *FOS* closely aligns with the offline approach *EOPS* and consistently outperforms *PBOS* for both T_c and ρ, as depicted in Figures 17(a) and 17(b), respectively. We compare the existing online synthesis approach *PBOS* [9], and the proposed *FOS* with *EOPS* [23] to evaluate the solution quality. The motive is not to show that our method outperforms *EOPS*, but rather to demonstrate that, as an online approach, *FOS* achieves results much closer to the highly optimized offline method than *PBOS* does. This highlights the effectiveness of the proposed work in delivering near-offline quality solutions within an online framework, which makes it more practical for real bioassay execution in a general-purpose setting.

In the case of MEDA, with an increasing mixer size constraint m, a variety of mixer sizes are possible, which affects the fragmented area A_f. Hence, by varying m from 4 to 16 in the interval of 3 (with sets of possible mixer-sizes as {4}, {4, 7}, {5, 7, 10}, {4, 6, 13} and {4, 6, 16}), we consider 50 random testcases for each mixer size constraint $m = 4, 7, 10, 13, 16$ (total 250 testcases). The average fragmented area (A_f) is compared among *EOPS* [23], *PBOS* [9], and *FOS*. It is observed that increasing m amplifies the fragmentation issue with *PBOS* as depicted in Figure 17(c). The proposed *FOS* addresses the fragmentation issue through *Compaction*, while *EOPS* is an offline approach that operates without a virtual topology, thereby eliminating any fragmentation issues. Moreover, we consider 7 real-life bioassays and compare the performance of *FOS* over *EOPS* and *PBOS* in terms of T_c, ρ, and A_f. Table 4 presents that *FOS* consistently outperforms *PBOS* and follows *EOPS* for these real-life bioassays as well.

We evaluated 150 random testcases from the testcase group $\mathcal{G}_i$ with varying placement row widths to compare the average number of routing sub-tasks, routing length, routing time, the number of times compaction was invoked, compaction length, and the time required for executing compaction. For the purpose of this comparison, we adopted the experimental conditions reported

Table 3. Description of Testcases in Figures 17 and 18

Testcases for MEDA†			Transformed testcases for DMFB		
Testcases	**Ratio Sum**	**$h \rightarrow$ (Mixers Used)**	**Transformed Testcases**	**Converted Ratio Sum**	**$h' \rightarrow$ (Mixers Used)**
$\mathcal{G}_1$	294	$3 \rightarrow (6,7,7)$	$\mathcal{G}'_1$	512	$9 \rightarrow [2,2,2,2,2,2,2,2,2]$
$\mathcal{G}_2$	144	$4 \rightarrow (2,3,4,6)$	$\mathcal{G}'_2$	512	$9 \rightarrow (2,2,2,2,2,2,2,2,2)$
$\mathcal{G}_3$	324	$5 \rightarrow (2,3,3,3,6)$	$\mathcal{G}'_3$	512	$9 \rightarrow (2,2,2,2,2,2,2,2,2)$
$\mathcal{G}_4$	144	$6 \rightarrow (2,2,2,2,3,3)$	$\mathcal{G}'_4$	512	$9 \rightarrow (2,2,2,2,2,2,2,2,2)$
$\mathcal{G}_5$	288	$7 \rightarrow (2,2,2,2,2,3,3)$	$\mathcal{G}'_5$	512	$9 \rightarrow (2,2,2,2,2,2,2,2,2)$

†Same testcases are used in Figure 17(a) and 17(b).

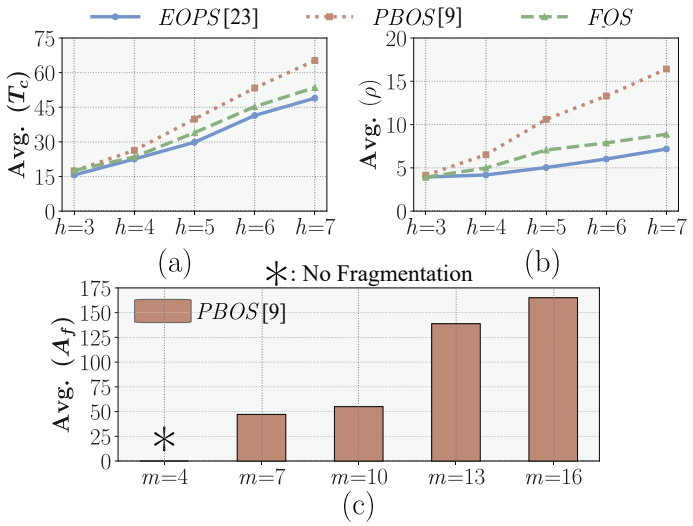


Fig. 17. Comparisons of average (a) T_c, (b) ρ and (c) A_f for *FOS* with *EOPS* [23], *PBOS* [9].

Table 4. Comparison of T_c, ρ, and A_f Among *EOPS* [23]‡, *PBOS* [9]‡, and *FOS* for Real-Life Bioassays

S.No.	Real-life Bio-assay	# V	*EOPS* [23]			*PBOS* [9]			*FOS*		
			T_c	ρ	A_f	T_c	ρ	A_f	T_c	ρ	A_f
1	Colorimetric Protein Assay (1:31:32) [26]	3	14.06	3	0	14.42	3	25	14.26	3	0
2	ChIP Lysis Buffer (67:38:3:67:14:67) [27]	8	19.10	4	0	21.75	6	52	20.52	5	0
3	Plasmid DNA (57:28:6:6:6:3:150) [28]	8	19.11	4	0	21.92	5	52	20.87	4	0
4	Splinkerette PCR (102:26:3:3:122) [29]	7	19.11	4	0	21.13	5	45	20.52	5	0
5	Silver Restriction Digest (180:26:26:5:5:14) [30]	8	19.10	4	0	21.60	6	52	21.00	5	0
6	Glucose Assay (1:319) [31]	3	13.06	3	0	13.35	3	19	13.20	3	0
7	RNA extraction from worms (243:13) [32]	4	20.09	4	0	20.14	5	25	19.54	5	0

‡Tuned for MEDA.

by [20]. Specifically, we considered square-shaped microelectrodes with an applied voltage of 100 V_{p-p}, an electrode gap of 100 μm, and a driving frequency of 1 kHz. Under these conditions, a droplet requires approximately 183 ms to traverse a single electrode. Table 5 shows that as the row width increases from 30 to 60, both routing time and compaction time increase slightly. It can

Table 5. Comparison Among Average Number of Routing Sub-Task, Routing Length, Routing Time, Average Number of Time Compaction Invoked, Compaction Length, and Time Required to Perform Compaction

Placement row width	# Routing sub-task	Routing length	Routing time (sec.)	# Compaction invoked	Compaction length	Compaction time (sec.)
30	8.33	214.83	39.31	1.10	11.26	2.06
40	7.26	244.60	44.76	1.23	15.20	2.78
50	6.26	270.16	49.44	1.20	21.26	3.89
60	5.46	275.73	50.49	1.06	23.46	4.29

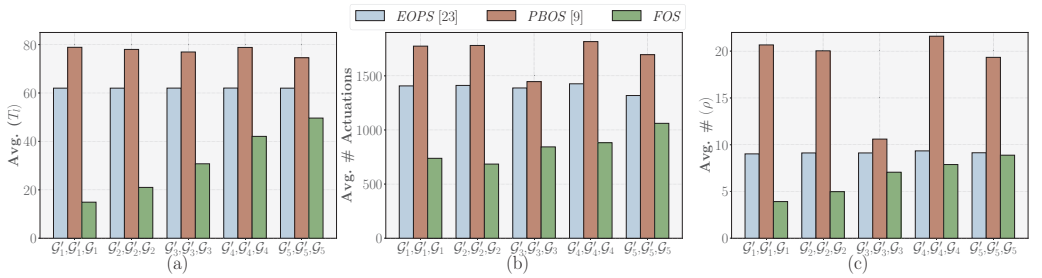


Fig. 18. Comparisons of average (a) scheduling length (T_l), (b) number of electrode actuations, and (c) number of routing sub-problems (ρ) among *EOPS* [23], *PBOS* [9] with testcases group $\mathcal{G}'_i$ and *FOS* with testcases group $\mathcal{G}_i$.

be observed that the average number of times compaction is invoked is smaller than the average number of routing sub-tasks. Moreover, the total compaction length is significantly smaller than the total routing length. As a result, the average time required for compaction is much smaller than the average time required for routing. Therefore, the overhead of the compaction step is negligible relative to routing. This demonstrates that the proposed *compaction* procedure improves resource utilization while introducing negligible overhead to the overall time required for assay completion.

7.2 Performance of *FOS* over DMFB-Based *EOPS* [23] and *PBOS* [9]

If *EOPS* and *PBOS* are not tuned for MEDA, they can avoid the fragmentation issue, but their performance parameters are affected. Hence, the mixing graphs determined for MEDA by *STWM* ($\mathcal{G}_i$s in Table 3) need to be transformed to DMFB-compatible mixing graphs with only two-size mixers. This process is explained in Figure 2, which transforms the mixing graphs by converting the initial ratio sum and the height of the mixing trees to get the corresponding 250 DMFB-compatible testcases ($\mathcal{G}'_i$s as shown in Table 3).

Figure 18 presents the comparative results for the same testcases ($\mathcal{G}_i$s for *FOS* and $\mathcal{G}'_i$s for *EOPS* and *PBOS*). The average scheduling length (T_l), the average number of electrode actuation, and the average number of routing sub-problems (ρ) are compared among *EOPS*, *PBOS*, and *FOS*, where *FOS* always outperforms *EOPS* and *PBOS* as depicted in Figure 18(a), 18(b), and 18(c), respectively. It is also observed that as the height difference between $\mathcal{G}_i$s and $\mathcal{G}'_i$s decreases, the performance gap narrows between *EOPS* and *FOS*, and it implies that for testcases with smaller height h and larger mixer size constraint m, *FOS* takes advantage of the variety of the MEDA-supported mixers and produces better solutions.

7.3 Comparison Among *WLSM* [5], *RLPM* [6], *AOP* and *AOP+PRE*

The overlapping value Φ needs to be minimized to enhance the reliability of the chip, which may increase the occupied chip-area α. We have evaluated 12 bioassays by varying the number of

Table 6. Comparison of Algorithmic Runtime (T_{cpu}), Occupied Area (α), and Maximum Overlapping Value (Φ) Among *WLSM* [5], *RLPM* [6], the Proposed *AOP* and *AOP+PRE* for 12 Testcases on a 6 × 72 MEDA Biochip

S.No.	Testcases	#V	T_l	*WLSM* [5]			*RLPM* [6]			*The proposed placer AOP with reliability enhancer PRE*					
										AOP			*AOP+PRE*		
				T_{cpu} (sec.)	α	Φ	T_{cpu} (sec.)	α	Φ	T_{cpu} (sec.)	α	Φ	T_{cpu} (sec.)	α	Φ
1	Polymerase Chain Reaction (PCR) [26]	7	18	1.03	252	1	142.83	252	1	0.61	144	3	0.98	252	1
2	In-Vitro Diagnostics (IVD) [26]	13	18	1.39	396	2	344.61	264	2	0.63	216	3	0.87	324	2
3	Colorimetric Protein Assay (CPA) [26]	56	54	3.25	432	7	2701.92	324	7	1.71	288	8	2.59	324	7
4	Synthetic Benchmark I (SB I) [6]	10	23	1.09	408	2	309.79	360	2	0.89	168	4	2.02	360	2
5	Synthetic Benchmark II (SB II) [6]	10	25	1.07	360	2	320.42	324	2	0.72	216	5	1.30	348	2
6	Synthetic Benchmark III (SB III)	13	27	1.44	432	2	433.06	348	2	0.81	216	3	1.74	408	2
7	Synthetic Benchmark IV (SB IV)	13	25	1.48	420	3	313.13	288	3	0.81	330	5	1.74	390	3
8	Synthetic Benchmark V (SB V)	15	16	1.78	408	2	357.79	312	2	0.72	288	3	1.05	354	2
9	Synthetic Benchmark VI (SB VI)	16	23	1.83	396	2	363.47	336	2	0.72	216	4	1.72	360	2
10	Synthetic Benchmark VII (SB VII)	18	22	1.48	408	3	342.69	300	3	0.89	300	4	1.02	348	3
11	Synthetic Benchmark VIII (SB VIII)	19	28	1.63	414	3	595.70	258	3	0.81	216	5	1.77	324	3
12	Synthetic Benchmark IX (SB IX)	22	40	1.89	420	3	556.49	288	3	1.12	216	7	3.10	408	3

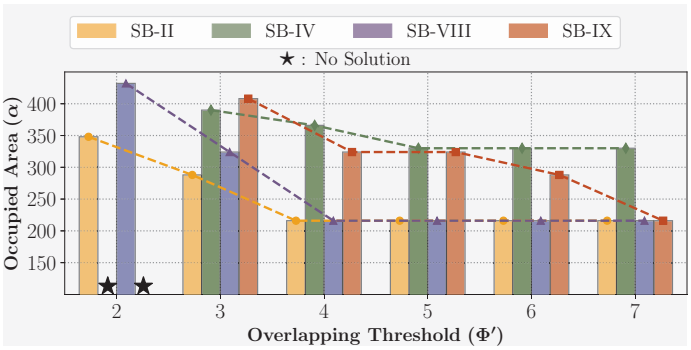


Fig. 19. Variation of α for Φ' in the range [2, 7] using *AOP+PRE*. Results are shown for synthetic benchmarks SB-(II, IV, VIII, IX), generated using the *STWM* [16] sample preparation method.

mixing nodes in corresponding DAGs, including both real-life and synthetic testcases. Algorithmic runtime (T_{cpu}), occupied chip-area (α), and overlapping value (Φ) are considered as performance parameters to compare the state-of-the-art placement methods *WLSM* [5] and *RLPM* [6] with *AOP* and *AOP+PRE*. We consider the same scheduling results for all the methods to have a fair comparison. *WLSM* [5] primarily focuses on reducing Φ without considering α, whereas *RLPM* [6] aims to reduce both α and Φ. However, it suffers from a higher T_{cpu} due to its adoption of a learning-based approach. *AOP* outperforms *WLSM* [5] and *RLPM* [6] in terms of both T_{cpu} and α, albeit with a penalty in reliability. In contrast, *AOP+PRE* slightly increases α to produce reliable solutions, where the resulting α is less than *WLSM* in all the testcases and is slightly increased compared with *RLPM* for a few testcases as depicted in Table 6. Despite producing reliable placement results with minimized occupied area, *RLPM* is unsuitable for online synthesis due to its higher T_{cpu}. Hence, *AOP* and *AOP+PRE* serve their purposes by producing area-efficient and reliable placement results in lower T_{cpu}, which is essential for the online synthesis of MEDA chips. If reliability is not the primary concern, we can increase ϕ' to reduce the chip-area α. Hence, this tradeoff between ϕ' and α can be noticed from Figure 19 as well, where for 4 testcases, we found that a smaller chip-area can be achieved by increasing ϕ' from 2 to 7.

8 Conclusions

Programmability and automation are crucial for fully utilizing the advanced functionalities of biochips. In this work, we have designed a dedicated virtual topology for MEDA biochips and an AOP placer with a reliability enhancer. The proposed online synthesis framework enables fast,

real-time, and cyber-physical compilation on MEDA. In the future, we can extend this work towards designing a full-fledged compiler for MEDA biochips, thereby enhancing their programmability to facilitate the efficient implementation of bioassays.

References

[1] J. H. Noh, J. Noh, E. Kreit, J. Heikenfeld, and P. D. Rack. 2012. Toward active-matrix lab-on-a-chip: Programmable electrofluidic control enabled by arrayed oxide thin film transistors. *Lab Chip* 12, 2 (2012), 353–360.

[2] K. Chakrabarty and F. Su. 2007. *Digital Microfluidic Biochips: Synthesis, Testing and Reconfiguration Techniques*. CRC Press.

[3] F. Su and K. Chakrabarty. 2008. High-level synthesis of digital microfluidic biochips. *ACM JETC* 3, 4 (2008), 1:1–1:32.

[4] T. Loveless, J. Ott, and P. Brisk. 2021. Time-and resource-constrained scheduling for digital microfluidic biochips. In *Proc. of the ICCPS* (2021), 198–208.

[5] Z. Zhong, T. C. Liang, and K. Chakrabarty. 2021. Enhancing the reliability of MEDA biochips using IJTAG and wear leveling. *IEEE TCAD* 40, 10 (2021), 2063–2076.

[6] D. Kundu, G. S. Vamsi, K. Vivek Veman, G. Mahidhar, and S. Roy. 2023. Reinforcement learning based module placement for enhancing reliability of MEDA digital microfluidic biochips. In *Proc. of the GLSVLSI*. 509–514.

[7] M. Elfar, Y. C. Chang, H. H. Y. Ku, T. C. Liang, K. Chakrabarty, and M. Pajic. 2023. Deep reinforcement learning-based approach for efficient and reliable droplet routing on MEDA biochips. *IEEE TCAD* 42, 4 (2023), 1212–1222.

[8] C. Curtis, D. Grissom, and P. Brisk. 2018. A compiler for cyber-physical digital microfluidic biochips. In *Proc. of the CGO* (2018), 365–377.

[9] D. Grissom and P. Brisk. 2014. Fast online synthesis of digital microfluidic biochips. *IEEE TCAD* 33, 3 (2014), 356–369.

[10] D. Grissom, C. Curtis, and P. Brisk. 2014. Interpreting assays with control flow on digital microfluidic biochips. *JETC* 10, 3 (2014), 1–30.

[11] M. Alistar and P. Pop. 2014. Online synthesis for operation execution time variability on digital microfluidic biochips. In *Proc of the ISIC* (2014), 356–359.

[12] T. Loveless, J. Ott, and P. Brisk. 2020. A performance-optimizing compiler for cyber-physical digital microfluidic biochips. In *Proc of the CGO* (2020), 171–184.

[13] W. Thies, J. P. Urbanski, T. Thorsen, and S. Amarasinghe. 2008. Abstraction layers for scalable microfluidic biocomputing. *Natural Computing* 7, 2 (2008), 255–275.

[14] S. Roy, B. B. Bhattacharya, S. Ghoshal, and K. Chakrabarty. 2014. Theory and analysis of generalized mixing and dilution of biochemical fluids using digital microfluidic biochips. *ACM JETC* 11, 1 (2014), 2.1–2.33.

[15] T. C. Liang, Y. S. Chan, T. Y. Ho, K. Chakrabarty, and C. Y. Lee. 2019. Sample preparation for multiple-reactant bioassays on micro-electrode-dot-array biochips. In *Proc. of the ASP-DAC* (2019), 468–473.

[16] D. Kundu and S. Roy. 2022. MEDA biochip based single- target fluidic mixture preparation with minimum wastage. In *Proc. of the DSD* 2022 (2022), 809–814.

[17] E.J. Griffith, S. Akella, and M. K. Goldberg. 2006. Performance characterization of a reconfigurable planar-array digital microfluidic system. *IEEE TCAD* 25, 2 (2026), 345–357.

[18] D. Grissom and P. Brisk. 2012. Path scheduling on digital microfluidic biochips. In *Proc. of the DAC* (2012), 26–35.

[19] P. Roy, H. Rahaman, and P. Dasgupta. 2010. A novel droplet routing algorithm for digital microfluidic biochips. In *Proc. of the GLSVLSI* (2010), 441–446.

[20] G. Wang, D. Teng, and S. K. Fan. 2011. Digital microfluidic operations on micro-electrode dot array architecture. *IET Nanobiotechnology* 5, 4 (2011), 152–160.

[21] B. Hadwen, G. R. Broder, D. Morganti, A. Jacobs, C. Brown, J. R. Hector, Y. Kubota, and H. Morgan. 2012. Programmable large area digital microfluidic array with integrated droplet sensing for bioassays. *Lab Chip* 12, 18 (2012), 3305–3313.

[22] T. C. Liang, Z. Zhong, M. Pajic, and K. Chakrabarty. 2020. Extending the lifetime of MEDA biochips by selective sensing on microelectrodes. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 3531–3543.

[23] N. Mohammadzadeh, R. Wille, and O. Keszocze. 2021. Efficient one-pass synthesis for digital microfluidic biochips. *ACM TODAES* 26, 4 (2021), 789–798.

[24] O. Keszocze, R. Wille, T. Y. Ho, and R. Drechsler. 2014. Exact one-pass synthesis of digital microfluidic biochips. In *Proc. of the 51st ACM/EDAC/IEEE Design Automation Conference (DAC'14)* (2014), 1–6.

[25] O. Keszocze, R. Wille, and R. Drechsler. 2020. One-pass synthesis for digital microfluidic biochips: A survey. In *Proc. of the ISDCS* (2020), 1–6.

[26] W. H. Minhass, P. Pop, and J. Madsen. 2012. System-level modeling and synthesis techniques for flow-based microfluidic very large scale integration biochips. *Technical Report, DTU Compute*, 2012.

[27] A Beginners Guide to ChIP. 2024. Retrieved from https://docs.abcam.com/pdf/chromatin/A-beginners-guide-to-ChIP.pdf. [Accessed: October, 2025].
[28] M. R. Green and J. Sambrook. 2016. Preparation of plasmid DNA by alkaline lysis with sodium dodecyl sulfate: Minipreps. *Cold Spring Harbor Protocols* 2016, 18 (2016), 911–916.
[29] A. G. Uren, H. Mikkers, J. Kool, L. van der Weyden, A. H. Lund, C. H. Wilson, R. Rance, J. Jonkers, and M. van Lohuizen. 2009. A high throughput splinkerette-PCR method for the isolation and sequencing of retroviral insertion sites. *Nature Protocols* 4, 5 (2009), 789–798.
[30] Openwetware. 2009. Retrieved from https://openwetware.org/wiki/Main_Page. [Accessed: October, 2025].
[31] ScienCell Research Laboratories. 2020. Glucose Assay. Retrieved from https://www.sciencellonline.com/PS/8418.pdf
[32] S. Poddar, S. Ghoshal, K. Chakrabarty, and B. B. Bhattacharya. 2016. Error-correcting sample preparation with cyber-physical digital microfluidic lab-on-chip. *ACM TODAES* 22, 1 (2016), 1–29.

Received 26 June 2025; revised 22 September 2025; accepted 15 October 2025